
MÉMOIRE

Présenté à

Institut Supérieur d'Informatique de Mahdia

Effectué à

Digi Growing

En vue de l'obtention du diplôme de

Licence Computer Engineering : Systèmes Embarqués et Internet des Objets

Élaboré par :

HADDAD FEDI , CHETOUI AHMED

Développement d'un robot semi-humanoïde modulaire intelligent basé sur l'IA et la vision artificielle

Soutenu le : Juin 2026

devant le jury composé de :

Président :

Rapporteur :

Encadreur Académique : Hamdi Aloulou

Encadreur Professionnel : Chawki Gharsellaoui

Année universitaire 2025–2026

Table des matières

Introduction Générale	1
1 Étude de Projet	3
Introduction	3
1.1 Présentation de l'organisme d'accueil - Digi Growing	4
1.2 Champ d'application	4
1.3 Problématique	5
1.4 Objectifs du projet	6
1.5 Étude et critique de l'existant	6
1.5.1 Étude de l'existant	6
1.5.2 Critique de l'existant	10
1.5.3 Solution proposée	11
1.6 Méthodologie adoptée	11
1.6.1 Fondements de la méthodologie Agile	12
1.6.2 Approches dérivées de l'Agile	12
1.6.3 Justification du choix pour AXEL	15
1.6.4 Application pratique dans le projet AXEL	15
Conclusion	15
2 Analyse et spécification des besoins	17
Introduction	17
2.1 Spécification des besoins	17
2.1.1 Identification des acteurs	18
2.1.2 Besoins fonctionnels	18
2.1.3 Besoins non fonctionnels	19
2.1.4 Diagramme de cas d'utilisation global	19
2.2 Gestion du projet avec l'approche Scrumban	20
2.2.1 Planification des sprints	22
2.3 Contraintes du système	23

2.4	Environnement logiciel	24
2.5	Environnement matériel	27
2.6	Architecture générale du système	31
2.7	Étude de l'architecture logicielle	33
2.7.1	Organisation des modules logiciels	33
2.7.2	Architecture basée sur Robot Operating System 2 (ROS2)	34
2.7.3	Justification du choix de ROS2	34
	Conclusion	35
3	Intégration matérielle et tests expérimentaux	36
	Introduction	36
3.1	Organisation du sprint	36
3.1.1	Backlog du Sprint 1	36
3.2	Installation du système d'exploitation	37
3.2.1	Choix d'Ubuntu 24.04 LTS	37
3.2.2	Procédure d'installation	38
3.3	Installation et configuration de ROS2 Jazzy	38
3.3.1	Présentation de ROS2 Jazzy Jalisco	38
3.3.2	Procédure d'installation de ROS2 Jazzy	39
3.4	Configuration de l'ESP32	39
3.4.1	Rôle de l'ESP32 dans le système AXEL	39
3.4.2	Configuration via Arduino IDE	40
3.5	Conception mécanique et modélisation 3D de la tête	40
3.5.1	Architecture mécanique de la tête	40
3.5.2	Modèle 3D et intégration des servomoteurs	41
3.6	Câblage et intégration matérielle	43
3.6.1	Architecture de câblage	43
3.6.2	Connexion des modules PCA9685	43
3.6.3	Connexion des servomoteurs	43
3.7	Contrôle des mouvements de la tête	45
3.7.1	Mécanisme de rotation de la tête	45
3.7.2	Synchronisation labiale par RMS	45
3.8	Sécurité et protection du système	46
3.8.1	Limites mécaniques des servomoteurs	46
3.8.2	Gestion énergétique et protection thermique	47
3.8.3	Analyse des limitations et perspectives d'amélioration	47
3.9	Tests de communication I2C	48
3.9.1	Validation de la communication	48
3.9.2	Résultats des tests	48

Conclusion	50
4 Interaction vocale et traitement du langage naturel	51
Introduction	51
4.1 Backlog du Sprint 2	51
4.2 Architecture du module vocal	53
4.2.1 Paramétrage du VAD serveur	53
4.2.2 Choix du Voice Activity Detection (VAD)	53
4.2.3 Vue d'ensemble du pipeline audio	54
4.2.4 Fonctionnement global du pipeline vocal	55
4.2.5 Description des composants	56
4.3 Intégration du microphone Spektra	56
4.4 Développement du nœud de perception ROS2	56
4.5 Intégration de l'API Gemini	57
4.5.1 Logique de décision et interprétation des commandes	58
4.5.2 Interaction contextuelle basée sur l'identification de l'utilisateur . .	59
4.5.3 Profils d'outils	61
4.5.4 Gestion du multilinguisme	61
4.5.5 Optimisation de la latence : Warm-up de session	62
4.6 Intégration du haut-parleur USB	63
4.7 Tests et validation	63
Conclusion	64
Bibliographie	65

Table des figures

1.1	Logo DigiGrowing	4
1.2	Robot humanoïde NAO	7
1.3	Robot humanoïde Pepper	8
1.4	Robot humanoïde Atlas	9
1.5	Robot humanoïde Unitree G1	9
1.6	Cycle itératif d'une approche Agile [8]	12
1.7	Organisation itérative selon Scrum [9]	13
1.8	Gestion visuelle du flux selon Kanban [10]	14
1.9	Approche hybride Scrumban [11]	14
2.1	Diagramme de cas d'utilisation global du robot AXEL	20
2.2	Logo de Visual Studio Code	24
2.3	Logo de GitHub	24
2.4	Logo de ROS2	24
2.5	Logo de Arduino IDE	24
2.6	Logo de Ubuntu	25
2.7	Logo de Python	25
2.8	Logo du langage C++	25
2.9	Bibliothèque React	25
2.10	Framework Electron	25
2.11	Bibliothèque Three.js	26
2.12	Langage TypeScript	26
2.13	Logo de draw.io	26
2.14	Logo de Overleaf	26
2.15	Logo de RViz	27
2.16	Logo de MoveIt	27
2.17	Raspberry Pi 5	27
2.18	Microcontrôleur ESP32	28
2.19	Driver PWM PCA9685	28

2.20	Servomoteur SG90	28
2.21	Servomoteur MG90S	29
2.22	Servomoteur MG996R	29
2.23	Servomoteur DS3218 (20 kg, 180°)	29
2.24	Servomoteur DS3225 (25 kg, 180°)	30
2.25	Microphone Spektra	30
2.26	Haut-parleur USB	30
2.27	Filament Hyper PLA RFID utilisé pour l'impression 3D	31
2.28	Architecture générale du robot AXEL	32
2.29	Architecture Logicielle du robot AXEL (basee sur ros2)	34
3.1	Système d'exploitation Ubuntu 24.04 LTS	38
3.2	Framework ROS2 Jazzy Jalisco	39
3.3	Environnement Arduino IDE pour la programmation de l'ESP32	40
3.4	Modèle 3D détaillé de la tête du robot AXEL	42
3.5	Architecture matérielle de la tête du robot AXEL	44
3.6	Prototype de la tête du robot AXEL après intégration des servomoteurs	49
4.1	Positionnement de la couche audio dans l'architecture globale du robot AXEL	52
4.2	Architecture du pipeline d'interaction vocale du robot AXEL	55
4.3	Architecture de communication avec l'API Gemini Live [24]	58
4.4	Diagramme d'activité du processus d'identification et d'interaction intelligente du robot AXEL	60
4.5	Implémentation du mécanisme de warm-up de session	62

Liste des tableaux

1.1	Comparaison des principales plateformes humanoïdes	10
2.1	Identification des acteurs du système	18
2.2	Besoins non fonctionnels	19
2.3	Extrait du Product Backlog du projet AXEL	21
2.4	Extrait du Product Backlog du projet AXEL (Partie 2)	22
2.5	Planification des sprints du projet AXEL	23
3.1	Backlog du Sprint 1 — Intégration matérielle et tests expérimentaux . . .	37
4.1	Backlog du Sprint 2 — Interaction vocale	52
4.2	Paramètres du VAD serveur et impact sur la latence	53
4.3	Correspondance entre commandes vocales et actions	58
4.4	Profils d’outils et impact sur la latence	61
4.5	Impact du warm-up sur la latence	62
4.6	Évaluation des performances du module d’interaction vocale	63

Liste des abréviations et acronymes

CV Computer Vision (Vision par Ordinateur). 1, 3, 5, 11, 12

IA Intelligence Artificielle. 1, 3, 11, 15

IHM Interaction Homme–Machine. 1, 3, 11

IoT Internet of Things (Internet des Objets). 1

ISIMA Institut Supérieur d’Informatique de Mahdia. 4

NLP Natural Language Processing. 1, 3, 5, 11, 12, 59

PFE Projet de Fin d’Études. 1, 3, 4

R&D Recherche et Développement. 5, 11

RMS Root Mean Square. 45, 46

ROS2 Robot Operating System 2. ii, 10, 23, 24, 26, 27, 33, 34, 36–39, 48, 52, 54, 57, 64

RViz ROS Visualization (Outil de visualisation 3D pour les données robotiques sous ROS). 21, 23, 26, 34, 39

SE Systèmes Embarqués. 1

VAD Détection de l’activité vocale. 51, 54, 56, 57, 63, 64

Introduction Générale

Dans un paysage technologique en mutation constante, l'évolution de l'Industrie 4.0 vers l'Industrie 6.0 [5] marque un tournant décisif. Cette nouvelle ère, portée par l'omniprésence de Internet of Things (Internet des Objets) (IoT) et la puissance de Intelligence Artificielle (IA) , transforme les usines en écosystèmes hyper-connectés et autonomes. Dans ce contexte, la robotique cognitive s'impose désormais comme le catalyseur de cette révolution, où les robots ne se limitent plus à l'exécution de tâches répétitives, mais évoluent vers des systèmes capables de percevoir leur environnement, de prendre des décisions et d'interagir avec l'être humain de manière naturelle et efficace.

Au cœur de cette évolution, la robotique humanoïde représente l'un des défis les plus complexes et prometteurs. En adoptant une morphologie et des vecteurs d'interaction proches de l'homme, ces systèmes visent à briser les barrières de l'adaptabilité, s'inscrivant ainsi dans des domaines stratégiques tels que la santé connectée, la surveillance intelligente et l'assistance de proximité.

C'est dans cette perspective d'innovation de pointe que s'inscrit le présent Projet de Fin d'Études (PFE), dédié à la conception et à la réalisation de **AXEL**. Plus qu'un simple robot, **AXEL** est conçu comme une plateforme humanoïde multi-tâche et multi-domaine, orchestrée par une architecture hybride alliant IA et Systèmes Embarqués (SE) de haute performance. L'originalité de ce projet réside dans l'intégration synergique de plusieurs briques technologiques majeures : la Computer Vision (Vision par Ordinateur) (CV) pour la perception spatiale, la reconnaissance vocale pour une communication fluide, et une Interaction Homme-Machine (IHM) optimisée.

À l'échelle nationale, si l'intérêt pour la robotique est croissant, le développement de plateformes humanoïdes reste freiné par des barrières de coûts et de complexité d'intégration logicielle. Le projet **AXEL** ambitionne de répondre à cette problématique en proposant une solution modulaire et évolutive, capable de servir de base expérimentale pour la recherche académique et les applications industrielles locales.

L'objectif central de ce travail est de poser les fondements d'un robot polyvalent capable de percevoir son environnement via des algorithmes de CV [6], de comprendre les instructions humaines par Natural Language Processing (NLP) [7], et d'exécuter des ac-

tions complexes avec précision.

Ce rapport détaille ainsi le cycle complet de développement de la plateforme **AXEL**. Après une analyse rigoureuse de l'état de l'art et des besoins, nous exposerons la conception architecturale, le déploiement des modules d'intelligence artificielle, pour finir par la validation expérimentale des capacités multi-tâches du robot **AXEL**.

1

Chapitre

Étude de Projet

Introduction

Le présent chapitre est consacré à la présentation générale de notre projet **AXEL**, une plateforme robotique humanoïde intelligente, ainsi qu'à l'organisme d'accueil, Digi-Growing.

L'objectif de cette introduction est de définir le contexte global du projet, ses objectifs stratégiques et son rôle précurseur dans le domaine de la robotique cognitive. Nous aborderons les défis technologiques et les enjeux majeurs liés au développement des robots humanoïdes, en mettant particulièrement l'accent sur les avancées actuelles en matière de perception par CV, de communication via le NLP et de prise de décision autonome.

Enfin, nous exposerons les solutions innovantes envisagées pour répondre aux problématiques d'interactivité et de polyvalence, ainsi que la méthodologie de développement adoptée pour la réalisation de ce PFE.

Ce rapport détaille ainsi le cycle complet de conception de la plateforme **AXEL**. Après une analyse rigoureuse des besoins, nous présenterons l'architecture du système, le déploiement des modules d'IA et d'IHM, pour conclure par la validation expérimentale des capacités multi-tâches du robot **AXEL**.

1.1 Présentation de l'organisme d'accueil - Digi Growing

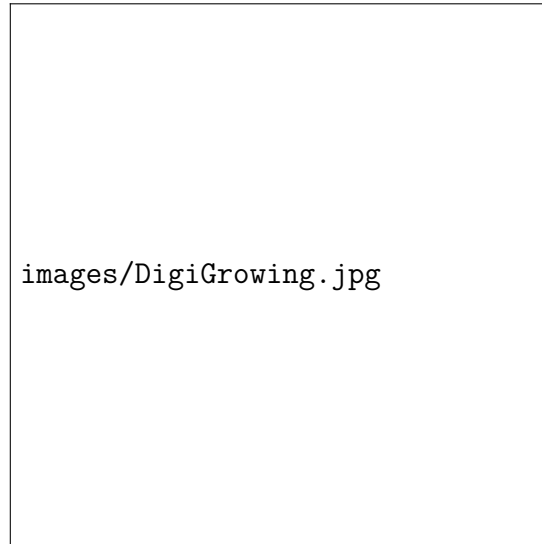


FIGURE 1.1 – Logo DigiGrowing

Le projet **AXEL** a été réalisé au sein de l'entreprise **DigiGrowing**. Il s'agit d'une entreprise tunisienne innovante, fondée en janvier 2024, spécialisée dans le développement de solutions technologiques intelligentes.

DigiGrowing intervient dans plusieurs domaines, notamment :

- les systèmes embarqués ;
- la robotique intelligente ;
- l'Internet des Objets (IoT) ;
- le développement web et mobile ;
- la formation et l'innovation technologique.

Grâce à son orientation vers les projets expérimentaux et innovants, DigiGrowing offre un cadre favorable à la conception de solutions complexes telles que les robots humanoïdes. Le projet **AXEL** s'inscrit pleinement dans cette dynamique.

1.2 Champ d'application

Le projet **AXEL** est une initiative académique développée dans le cadre d'un PFE en vue de l'obtention du diplôme de Licence en *Computer Engineering : Systèmes Embarqués et Internet des Objets* à l'Institut Supérieur d'Informatique de Mahdia (ISIMA). Il s'inscrit dans les domaines de pointe de la robotique humanoïde, de l'intelligence artificielle et des systèmes connectés.

Cette plateforme multidisciplinaire mobilise des compétences transversales en conception matérielle, en traitement de données (CV et NLP), ainsi qu'en développement d'interfaces interactives. Ce projet sera mené sur une période de quatre mois, débutant le **2 février** et s'achevant le **30 Mai 2026**.

1.3 Problématique

Dans un contexte mondial marqué par la transition vers l'Industrie 6.0, la robotique de service et d'assistance connaît une mutation sans précédent. Alors que les besoins en automatisation flexible et en interaction sociale augmentent — que ce soit pour l'assistance aux personnes, l'accueil ou la surveillance — le marché actuel reste dominé par des solutions onéreuses, fermées et hyper-spécialisées. Cette rigidité technologique freine considérablement le déploiement massif de robots capables de s'adapter dynamiquement à des environnements non structurés.

En Tunisie, ces défis sont exacerbés par un accès limité aux plateformes robotiques de pointe, souvent importées à des coûts prohibitifs. À cette barrière financière s'ajoute une **indisponibilité matérielle** chronique sur le marché local, rendant l'approvisionnement en composants électroniques spécifiques particulièrement complexe. Ce fossé technologique prive l'écosystème local d'outils expérimentaux nécessaires pour maîtriser les interactions entre l'Intelligence Artificielle et les systèmes embarqués. Le manque de solutions endogènes, modulaires et ouvertes contraint souvent l'innovation nationale à rester théorique, sans aboutissement matériel concret.

Notre projet s'attaque à cette rupture technologique en posant la question centrale suivante :

Comment concevoir une plateforme robotique humanoïde modulaire et abordable, capable d'orchestrer une interaction fluide homme-machine, de percevoir son environnement avec précision grâce à l'IA, tout en garantissant une exécution polyvalente de tâches complexes ?

Cette problématique vise à concilier trois impératifs majeurs :

- **L'accessibilité technologique**, en proposant une architecture matérielle et logicielle optimisée qui pallie l'**indisponibilité des composants** et réduit les barrières de coûts sans sacrifier la performance, démocratisant ainsi la robotique avancée.
- **La polyvalence opérationnelle**, en brisant le paradigme du robot mono-tâche grâce à une fusion intelligente entre CV et le NLP.
- **L'innovation locale**, en démontrant la capacité de l'ingénierie tunisienne à développer des systèmes autonomes sophistiqués, réduisant ainsi la dépendance aux solutions importées et stimulant la Recherche et Développement (R&D) nationale.

1.4 Objectifs du projet

L’objectif principal de ce projet est de concevoir et de développer **AXEL**, un robot humanoïde multi-tâche capable d’interagir avec son environnement et d’exécuter différentes fonctions de manière autonome ou semi-autonome.

Les objectifs spécifiques sont :

- concevoir une architecture matérielle adaptée au robot humanoïde ;
- développer une architecture logicielle modulaire ;
- intégrer des capacités de perception et de prise de décision ;
- permettre l’exécution de plusieurs tâches dans différents domaines ;
- assurer l’interaction avec l’utilisateur ;
- tester et valider le fonctionnement global du système.

1.5 Étude et critique de l’existant

1.5.1 Étude de l’existant

La robotique humanoïde a connu une évolution significative au cours des dernières décennies, portée par les avancées en mécatronique, en intelligence artificielle et en systèmes embarqués. Plusieurs plateformes humanoïdes ont été développées dans des contextes éducatifs, industriels et de recherche. Toutefois, chacune de ces solutions présente des objectifs spécifiques et des limitations techniques qui influencent leur champ d’application.

Robot NAO

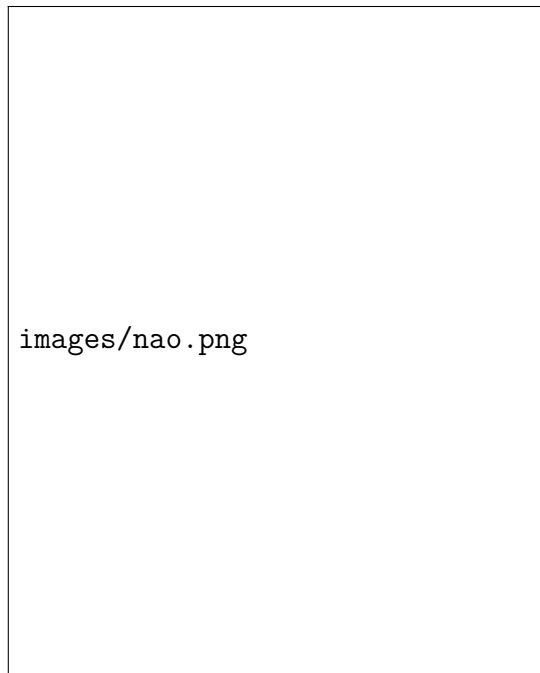


FIGURE 1.2 – Robot humanoïde NAO

NAO [1], conçu par SoftBank Robotics, est un robot humanoïde de taille réduite qui s’est imposé comme une référence incontournable dans les milieux éducatifs et de recherche à l’échelle mondiale. Doté d’une dotation sensorielle embarquée complète — comprenant des caméras, des microphones directionnels et des capteurs inertiels — il offre une capacité d’interaction basique mais fonctionnelle avec son environnement immédiat. Néanmoins, son architecture logicielle repose sur un écosystème propriétaire peu ouvert, ce qui restreint considérablement l’intégration d’algorithmes modernes d’apprentissage profond ou de vision artificielle. De surcroît, son tarif commercial élevé constitue un obstacle majeur à son adoption dans des contextes académiques à ressources limitées.

Robot Pepper



FIGURE 1.3 – Robot humanoïde Pepper

Pepper [2] est une plateforme humanoïde développée par SoftBank Robotics, pensée avant tout comme un agent conversationnel et d'accueil. Équipé d'un écran tactile frontal, de microphones directionnels et d'un système de reconnaissance des expressions faciales, il excelle dans les environnements commerciaux tels que les points de vente ou les espaces d'accueil. Cependant, cette spécialisation vers le relationnel constitue également sa principale limite : Pepper dispose d'une autonomie décisionnelle réduite et d'une mobilité contrainte, le rendant peu adapté à des scénarios exigeant une navigation complexe ou une interaction avec des environnements physiques non structurés.

Robot Atlas

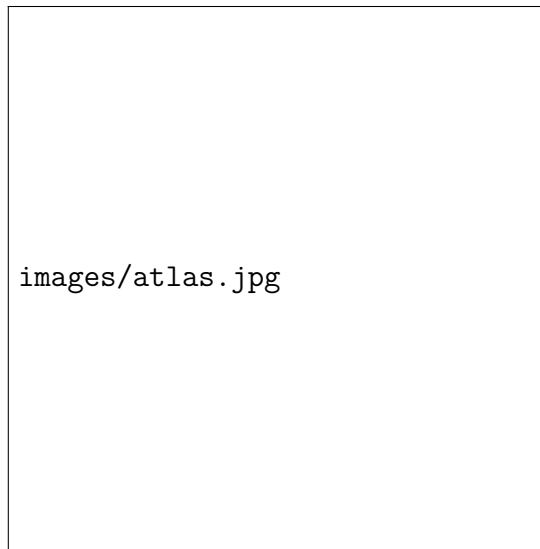


FIGURE 1.4 – Robot humanoïde Atlas

Atlas [3], conçu par Boston Dynamics, représente l'état de l'art en matière de robotique humanoïde mobile. Grâce à une architecture mécanique hautement articulée et des algorithmes de contrôle dynamique en temps réel, ce robot est capable d'évoluer dans des terrains accidentés, d'effectuer des sauts et de réaliser des manipulations d'objets avec une précision remarquable. Néanmoins, Atlas demeure une plateforme réservée à la recherche de très haut niveau : son coût de fabrication prohibitif, la complexité de sa maintenance et les infrastructures spécialisées qu'il nécessite le rendent totalement inaccessible dans un cadre académique standard, et a fortiori dans le contexte tunisien.

Robot Unitree G1

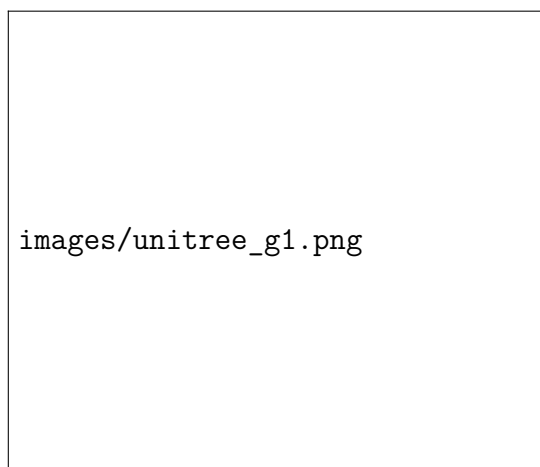


FIGURE 1.5 – Robot humanoïde Unitree G1

Le Unitree G1[4], conçu par Unitree Robotics, incarne une avancée majeure dans le domaine de la robotique humanoïde accessible. Avec ses 23 degrés de liberté couvrant l’intégralité de sa structure corporelle — bras, jambes et tronc — ce robot adopte une morphologie pleinement humanoïde lui permettant d’exécuter des mouvements élaborés : marche dynamique, transitions posturales complexes, relèvement autonome et manipulation d’objets. Contrairement aux plateformes propriétaires et fermées qui dominent le marché, le G1 intègre nativement le framework ROS2 et propose un SDK ouvert, offrant ainsi une flexibilité appréciable pour les environnements de recherche et de développement. Cependant, en dépit d’un positionnement tarifaire plus compétitif que des solutions telles qu’Atlas, son coût d’acquisition reste difficilement supportable dans un contexte académique à ressources limitées, et sa disponibilité sur le marché tunisien demeure très restreinte, ce qui freine considérablement son adoption locale.

TABLE 1.1 – Comparaison des principales plateformes humanoïdes

Critères techniques	NAO	Pepper	Atlas	Unitree G1
Mobilité humanoïde	Oui	Oui	Oui	Oui
Degré d’autonomie	Faible–modéré	Modéré	Élevé	Modéré–élevé
Capacités IHM (voix + vision)	Oui	Avancées	Limitées	Basiques
Capteurs de vision	Caméras HD	Caméras HD	Vision 3D	Caméras RGB-D
Capteurs de distance	Ultrasons	Lidar + ultrasons	Lidar avancé	IMU + encodeurs
Architecture logicielle	Partiellement fermée	Fermée	Propriétaire	Ouverte (ROS2)
Personnalisation algorithmique	Limitée	Limitée	Complexe	Accessible
Adaptation académique	Oui	Partielle	Non	Partielle
Coût estimé	Élevé	Très élevé	Extrêmement élevé	Modéré
Objectif principal	Éducation	Interaction sociale	Recherche	Recherche / Usage général

À travers cette analyse comparative, il apparaît que les solutions existantes répondent à des objectifs spécifiques mais ne proposent pas une plateforme simultanément modulaire, évolutive et économiquement adaptée au contexte académique.

1.5.2 Critique de l’existant

L’étude des plateformes humanoïdes actuelles met en évidence plusieurs limites structurelles :

- **Limite technique** : la majorité des robots sont conçus pour des tâches spécifiques, ce qui réduit leur polyvalence et leur capacité d’adaptation multi-domaine.
- **Limite architecturale** : les architectures matérielles et logicielles sont souvent fermées ou peu modulaires, limitant l’intégration de nouvelles fonctionnalités.
- **Limite économique** : le coût élevé de certaines plateformes rend leur acquisition difficile dans des environnements académiques.

- **Limite évolutive** : peu de solutions permettent une évolution progressive du système en fonction des besoins expérimentaux.

Ces limites démontrent la nécessité de concevoir une plateforme humanoïde flexible, extensible et adaptée aux contraintes académiques.

1.5.3 Solution proposée

Face aux barrières technologiques et économiques qui freinent le déploiement de la robotique humanoïde en Tunisie, notamment l'indisponibilité des composants propriétaires et les coûts prohibitifs des solutions importées, notre projet propose une plateforme robotique anthropomorphe, modulaire et cognitive. L'objectif est de démocratiser l'accès à la robotique de service en offrant une alternative performante, capable d'orchestrer des interactions complexes tout en s'affranchissant des contraintes matérielles locales.

Nous avons conçu une architecture systémique hybride, reposant sur une distribution intelligente de la puissance de calcul. Ce système intègre une unité de traitement dédiée à l'IA pour la perception de haut niveau, couplée à des microcontrôleurs temps réel pour le pilotage précis des actionneurs. Contrairement aux solutions monolithiques et fermées du marché, notre approche dissocie l'intelligence décisionnelle du contrôle moteur, permettant ainsi de contourner la pénurie de composants spécifiques par l'utilisation de hardware standardisé et accessible.

Pour une interaction homme-machine (IHM) fluide et immersive, nous avons développé une interface multimodale unifiée. Cette couche logicielle permet au robot de traiter simultanément les flux visuels et auditifs, offrant ainsi une compréhension contextuelle de l'environnement. Cela assure une communication naturelle avec l'utilisateur, transformant le robot d'un simple exécutant en un véritable assistant interactif.

L'un des aspects les plus innovants de notre solution réside dans la fusion sensorielle pilotée par l'intelligence artificielle. En corrélant les données issues de la Vision par Ordinateur (CV) et du Traitement du Langage Naturel (NLP), le système est capable d'identifier des objets, de comprendre des commandes vocales complexes et d'adapter son comportement en temps réel. Cette synergie permet à **AXEL** d'anticiper les besoins de l'utilisateur et d'exécuter des tâches multi-domaines avec une autonomie accrue.

Grâce à cette approche pragmatique et innovante, notre projet transforme la robotique humanoïde académique en une réalité tangible, fiable et reproductible, contribuant ainsi à l'essor de la R&D nationale et à la maîtrise des technologies de l'Industrie 6.0.

1.6 Méthodologie adoptée

Le développement de la plateforme humanoïde **AXEL** ne se limite pas à une simple implémentation technique ; il s'agit d'un processus progressif d'intégration entre intelli-

gence artificielle, électronique embarquée et architecture mécatronique. La nature multidisciplinaire du projet impose ainsi une organisation structurée, capable d’accompagner l’évolution continue du système.

Dans un environnement caractérisé par l’expérimentation, les ajustements matériels fréquents et l’optimisation progressive des algorithmes de CV et de NLP, l’adoption d’une méthodologie flexible s’est révélée indispensable. Pour répondre à ces exigences, nous avons retenu une approche Agile hybride fondée sur la méthodologie **Scrumban**.

1.6.1 Fondements de la méthodologie Agile

La méthodologie Agile repose sur un principe central : développer par itérations successives en favorisant l’adaptation continue. Contrairement aux modèles linéaires traditionnels, elle permet une évolution progressive du produit à travers des cycles courts et contrôlés.

Dans le cadre du projet AXEL, cette approche offre plusieurs avantages :

- une validation incrémentale des modules intelligents ;
- une réduction des risques liés à l’intégration matérielle ;
- une amélioration continue basée sur les tests expérimentaux ;
- une meilleure coordination entre perception, décision et action.

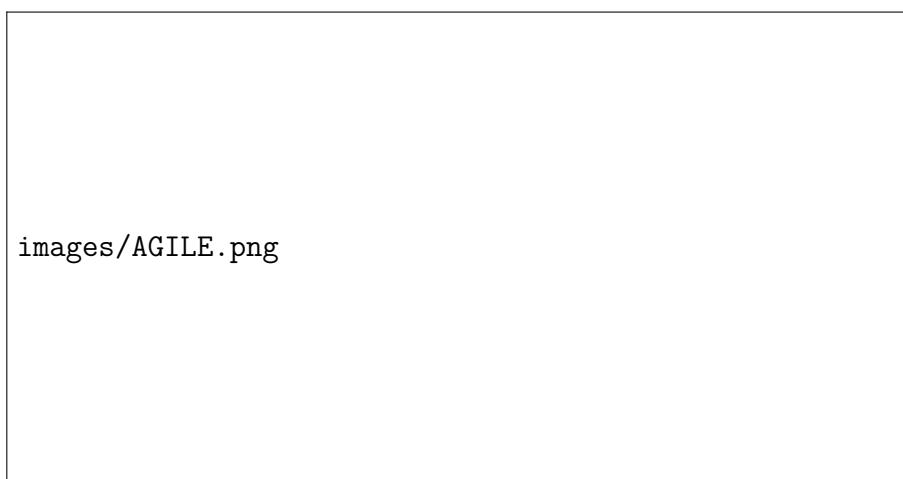


FIGURE 1.6 – Cycle itératif d’une approche Agile [8]

1.6.2 Approches dérivées de l’Agile

L’Agile se décline en plusieurs cadres méthodologiques adaptés à différents contextes de projet. Parmi eux, Scrum et Kanban figurent parmi les plus répandus.

Scrum

Scrum propose un cadre structuré reposant sur des cycles courts appelés **Sprints**. Chaque Sprint vise à produire un incrément fonctionnel du système.

Cette organisation présente l'avantage d'imposer un rythme régulier, particulièrement utile pour structurer le développement des sous-systèmes du robot (vision, commande moteur, interaction vocale).

Les principes fondamentaux de Scrum incluent :

- une planification périodique ;
- une répartition claire des responsabilités ;
- des points de synchronisation réguliers ;
- une évaluation continue des livrables.

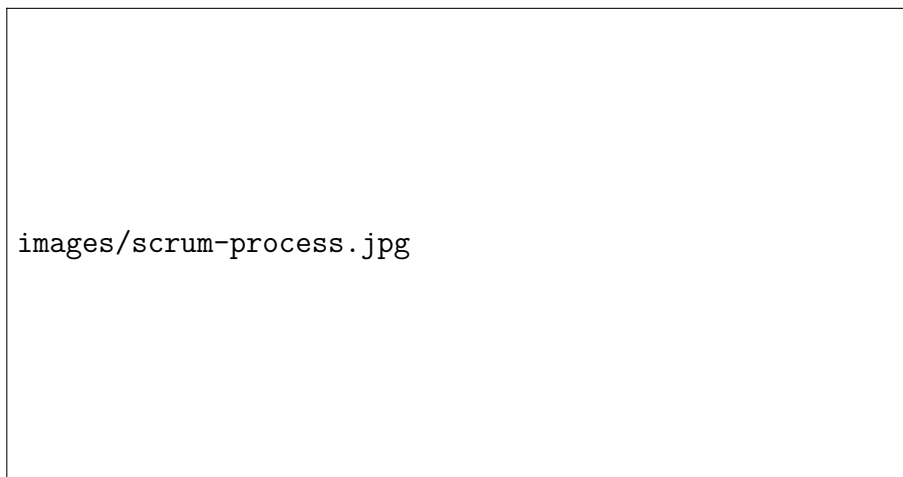


FIGURE 1.7 – Organisation itérative selon Scrum [9]

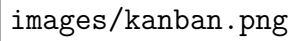
Kanban

Kanban adopte une logique différente en privilégiant un flux continu de tâches. L'accent est mis sur la visualisation du travail et la maîtrise de la charge en cours.

Cette approche est particulièrement pertinente pour un projet robotique où certaines tâches (intégration électronique, calibration des capteurs, tests moteurs) peuvent évoluer indépendamment du calendrier initial.

Ses principes clés sont :

- visualisation du flux de travail ;
- limitation du *Work In Progress (WIP)* ;
- ajustement dynamique des priorités ;
- amélioration continue du processus.



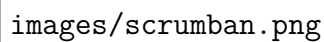
images/kanban.png

FIGURE 1.8 – Gestion visuelle du flux selon Kanban [10]

Scrumban

Scrumban représente une convergence entre la rigueur organisationnelle de Scrum et la souplesse opérationnelle de Kanban. Cette hybridation permet de conserver des cycles structurés tout en assurant une adaptation continue aux imprévus techniques.

Dans un projet tel que AXEL, où l'interaction entre composants logiciels et matériels est permanente, cette combinaison favorise un équilibre entre planification et réactivité.



images/scrumban.png

FIGURE 1.9 – Approche hybride Scrumban [11]

1.6.3 Justification du choix pour AXEL

Le choix de Scrumban s’est imposé en raison des caractéristiques propres au projet :

- intégration simultanée de modules intelligents et de composants physiques ;
- incertitudes liées aux performances des algorithmes ;
- dépendance à la disponibilité des équipements ;
- équipe restreinte nécessitant flexibilité et coordination étroite.

Scrum seul aurait imposé un cadre trop rigide face aux ajustements matériels fréquents, tandis que Kanban seul aurait manqué de structuration pour organiser les phases de validation.

Scrumban offre ainsi un compromis optimal, combinant discipline organisationnelle et adaptabilité technique.

1.6.4 Application pratique dans le projet AXEL

Concrètement, le développement a été organisé en cycles de deux semaines, chaque cycle visant l’intégration d’un sous-système fonctionnel (perception visuelle, commande vocale, contrôle moteur, fusion sensorielle).

Un tableau Kanban a été utilisé pour assurer :

- la visualisation des tâches en cours ;
- la limitation volontaire du nombre de développements simultanés ;
- la réévaluation continue des priorités ;
- le suivi précis de l’avancement global.

Cette organisation progressive a permis de transformer AXEL d’un prototype modulaire en une plateforme humanoïde cohérente et expérimentalement validée.

Conclusion

Ce premier chapitre a permis de situer le projet **AXEL** dans son contexte académique et technologique, en mettant en lumière les enjeux liés à la conception d’une plateforme humanoïde intelligente intégrant l’IA et les systèmes embarqués.

Après la présentation de l’organisme d’accueil et du champ d’application, nous avons défini la problématique centrale, puis analysé les limites des solutions existantes, justifiant ainsi la pertinence d’une architecture modulaire, évolutive et adaptée au contexte académique.

La méthodologie Agile retenue, implémentée à travers l’approche Scrumban, a permis d’assurer un développement progressif, structuré et maîtrisé du projet.

Le chapitre suivant sera consacré à la planification initiale et à la conception détaillée de l'architecture matérielle et logicielle de la plateforme **AXEL**.

2

Chapitre

Analyse et spécification des besoins

Introduction

Avant de procéder à la conception détaillée du robot humanoïde **AXEL**, il est essentiel d'identifier et d'analyser les besoins du système. Cette étape permet de définir les fonctionnalités attendues du robot ainsi que les contraintes techniques liées à sa réalisation.

L'analyse des besoins constitue une phase fondamentale dans le développement d'un système complexe, car elle permet de structurer les objectifs du projet et d'orienter les choix technologiques. Dans le cas du robot AXEL, cette analyse doit prendre en compte à la fois les aspects logiciels, matériels et les interactions avec l'utilisateur.

Dans ce chapitre, nous présentons la spécification des besoins du système, l'identification des acteurs impliqués ainsi que les besoins fonctionnels et non fonctionnels du robot. Ces éléments serviront de base pour la conception de l'architecture du système présentée dans le chapitre suivant.

2.1 Spécification des besoins

La spécification des besoins consiste à définir les fonctionnalités principales que doit offrir le robot humanoïde **AXEL**. Ces besoins représentent les attentes générales du système et permettent d'orienter la conception de l'architecture matérielle et logicielle.

Dans le cadre de ce projet, le robot AXEL doit être capable d'interagir avec son environnement et avec l'utilisateur à travers différents mécanismes d'intelligence artificielle et de contrôle embarqué.

Plus précisément, le système doit permettre :

- la perception de l’environnement à l’aide de capteurs et d’algorithmes de vision par ordinateur ;
- la compréhension des commandes vocales grâce aux techniques de traitement du langage naturel ;
- l’exécution d’actions physiques à travers le contrôle des actionneurs ;
- l’interaction avec l’utilisateur via une interface homme-machine intuitive ;
- l’intégration de plusieurs modules intelligents permettant au robot d’exécuter différentes tâches.

Ces besoins constituent le socle fonctionnel du système et guideront la conception détaillée de la plateforme robotique.

2.1.1 Identification des acteurs

Les acteurs représentent les entités qui interagissent directement ou indirectement avec le système. Dans le cadre du projet AXEL, plusieurs acteurs peuvent être identifiés selon leur rôle dans l’utilisation ou la gestion du robot.

Le tableau suivant présente les principaux acteurs du système.

TABLE 2.1 – Identification des acteurs du système

Acteur	Description
Utilisateur	Personne interagissant avec le robot à travers des commandes vocales ou des interactions directes.
Administrateur	Personne responsable de la configuration, de la maintenance et du suivi du fonctionnement du robot.

L’identification des acteurs permet de mieux comprendre les interactions possibles avec le système et de définir les fonctionnalités nécessaires à son fonctionnement.

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent les fonctionnalités que le robot AXEL doit fournir pour répondre aux attentes des utilisateurs. Ils définissent les services que le système doit assurer.

Les principaux besoins fonctionnels identifiés pour le robot AXEL sont les suivants :

- **BF1** : Le robot doit être capable de reconnaître les commandes vocales de l’utilisateur.
- **BF2** : Le robot doit analyser son environnement à l’aide de la vision par ordinateur.

- **BF3** : Le robot doit interagir avec l'utilisateur de manière naturelle.
- **BF4** : Le robot doit exécuter des actions physiques via ses actionneurs.
- **BF5** : Le robot doit traiter les informations collectées afin de prendre des décisions appropriées.
- **BF6** : Le système doit permettre l'intégration de nouveaux modules pour étendre les capacités du robot.

Ces besoins fonctionnels constituent les fonctionnalités principales du système et orientent la conception des différents modules du robot.

2.1.3 Besoins non fonctionnels

Les besoins non fonctionnels décrivent les caractéristiques de qualité du système. Ils définissent les contraintes liées à la performance, la fiabilité et la sécurité du robot.

Les principaux besoins non fonctionnels du système AXEL sont présentés dans le tableau suivant :

TABLE 2.2 – Besoins non fonctionnels

Besoin	Description
Performance	Le système doit offrir un temps de réponse rapide lors du traitement des commandes et des interactions.
Fiabilité	Le robot doit fonctionner de manière stable et continue pendant son utilisation.
Sécurité	Les interactions entre le robot et l'utilisateur doivent être sûres et contrôlées.
Modularité	L'architecture du système doit permettre l'ajout de nouveaux modules ou fonctionnalités.
Évolutivité	Le système doit pouvoir évoluer avec l'intégration de nouvelles technologies ou capacités.

2.1.4 Diagramme de cas d'utilisation global

Afin de représenter de manière claire les interactions entre les différents acteurs et le système, un diagramme de cas d'utilisation global est utilisé. Ce type de diagramme permet de modéliser les principales fonctionnalités offertes par le robot humanoïde **AXEL** ainsi que les interactions possibles avec les acteurs externes.

Le diagramme de cas d'utilisation constitue une représentation graphique des services fournis par le système. Il met en évidence les différentes actions que les acteurs peuvent

réaliser à travers l'utilisation du robot, telles que l'interaction vocale, l'analyse de l'environnement, l'exécution de commandes ou encore la gestion et la configuration du système.

Dans le cadre du projet **AXEL**, ce diagramme illustre principalement les interactions entre l'utilisateur, l'administrateur et le robot. Il permet ainsi de présenter de manière globale les fonctionnalités essentielles du système et de faciliter la compréhension de son fonctionnement général.

La figure suivante présente le diagramme de cas d'utilisation global du système **AXEL**.



FIGURE 2.1 – Diagramme de cas d'utilisation global du robot AXEL

2.2 Gestion du projet avec l'approche Scrumban

Dans le cadre du développement du robot **AXEL**, nous avons adopté l'approche **Scrumban**, qui combine les principes des méthodologies Scrum et Kanban. Cette approche permet de structurer le développement du projet tout en conservant une grande flexibilité dans la gestion des tâches.

L'utilisation de Scrumban facilite l'organisation du travail, la priorisation des fonctionnalités à développer et le suivi de l'avancement du projet à travers des cycles de développement appelés *sprints*.

TABLE 2.3 – Extrait du Product Backlog du projet AXEL

ID	User Story	Priorité
US1	En tant qu'utilisateur, je souhaite interagir avec le robot à l'aide de commandes vocales afin de lui transmettre des instructions.	Haute
US2	En tant qu'utilisateur, je souhaite que le robot analyse son environnement à l'aide de ses capteurs afin de détecter les obstacles et les objets.	Haute
US3	En tant qu'utilisateur, je souhaite que le robot réponde vocalement via le haut-parleur afin de confirmer l'exécution des commandes reçues.	Haute
US4	En tant qu'administrateur, je souhaite pouvoir contrôler directement le robot depuis l'interface de supervision afin d'envoyer des commandes manuelles.	Haute
US5	En tant qu'utilisateur, je souhaite que le robot traite les commandes vocales via l'IA afin de prendre des décisions adaptées au contexte.	Moyenne
US6	En tant qu'utilisateur, je souhaite que le robot effectue des expressions faciales adaptées afin de rendre l'interaction plus naturelle.	Moyenne
US7	En tant qu'utilisateur, je souhaite que le robot exécute des mouvements contrôlés afin d'interagir physiquement avec son environnement.	Moyenne
US8	En tant qu'administrateur, je souhaite visualiser l'état du robot via ROS Visualization (Outil de visualisation 3D pour les données robotiques sous ROS) (RViz) afin de superviser et déboguer son fonctionnement.	Moyenne
US9	En tant qu'administrateur, je souhaite surveiller les données des capteurs afin de vérifier le bon fonctionnement du système.	Basse
US10	En tant qu'administrateur, je souhaite configurer les modules IA afin d'adapter le comportement du robot aux différents scénarios.	Basse

TABLE 2.4 – Extrait du Product Backlog du projet AXEL (Partie 2)

ID	User Story	Priorité
US11	En tant qu'utilisateur, je souhaite que le robot puisse saisir et déplacer un objet afin d'effectuer une tâche de pick and place.	Haute
US12	En tant qu'utilisateur, je souhaite que le robot détecte automatiquement la position d'un objet sur une table afin de le manipuler correctement.	Haute
US13	En tant qu'utilisateur, je souhaite que le robot évite les collisions lors des mouvements du bras afin de manipuler les objets en toute sécurité.	Haute
US14	En tant qu'utilisateur, je souhaite que le robot reconnaisse les visages afin d'identifier les personnes présentes dans son environnement.	Moyenne
US15	En tant qu'utilisateur, je souhaite que le robot suive une personne afin d'assurer une interaction dynamique.	Moyenne
US16	En tant qu'utilisateur, je souhaite que le robot se déplace vers une destination précise afin d'assister dans différents espaces.	Moyenne
US17	En tant qu'administrateur, je souhaite pouvoir arrêter immédiatement le robot en cas d'urgence afin de garantir la sécurité.	Haute

2.2.1 Planification des sprints

Afin d'organiser le développement du projet, les différentes tâches identifiées dans le backlog ont été réparties en plusieurs sprints. Chaque sprint correspond à une période de développement durant laquelle un ensemble de fonctionnalités du robot AXEL est implémenté et testé.

La planification des sprints permet ainsi de structurer progressivement la réalisation du projet et de valider chaque module du système. Le développement du projet AXEL a été structuré en plusieurs sprints, permettant une implémentation progressive et maîtrisée des différentes fonctionnalités du robot.

Le tableau suivant présente la planification détaillée des sprints du projet.

TABLE 2.5 – Planification des sprints du projet AXEL

Sprint	Objectif	Tâches	Durée
Sprint 1	Intégration matérielle et tests expérimentaux	Installation d'Ubuntu 24.04 LTS sur Raspberry Pi 5, configuration de ROS2 Jazzy, configuration de l'ESP32 via Arduino IDE, câblage des modules PCA9685, intégration des servomoteurs (MG90, MG90S, MG996R), tests de communication I2C entre Raspberry Pi et ESP32, mise en place du dépôt GitHub. Livrable : Prototype de la tête du robot AXEL intégrant les servomoteurs et l'architecture de contrôle.	3 semaines
Sprint 2	Audio et interaction vocale	Intégration du microphone USB, développement du nœud de perception ROS2, implémentation de la capture audio en temps réel, connexion et configuration de l'API Gemini, traitement NLP des commandes vocales, génération des réponses intelligentes, intégration du haut-parleur USB, tests de reconnaissance et de synthèse vocale.	3 semaines
Sprint 3	Vision et intelligence artificielle	Intégration de la caméra, développement du module Computer Vision, implémentation de la détection et reconnaissance d'objets, connexion à l'API Gemini Robotics, développement du nœud IA ROS2, tests de perception visuelle et de décision autonome.	3 semaines
Sprint 4	Mouvements et actions du robot	Développement du nœud de contrôle moteur ROS2, intégration de MoveIt 2, calcul de la cinématique des bras, programmation des mouvements de la tête, pilotage des servomoteurs via PCA9685, visualisation dans RViz, calibration des positions, tests de fluidité et de précision.	3 semaines

2.3 Contraintes du système

La réalisation du robot humanoïde AXEL doit prendre en compte plusieurs contraintes techniques et matérielles.

Parmi les principales contraintes identifiées, on peut citer :

- la disponibilité limitée de certains composants électroniques sur le marché local ;
- les contraintes budgétaires liées à la réalisation du prototype ;
- les limitations en termes de puissance de calcul pour le traitement des algorithmes d'intelligence artificielle ;
- la complexité de l'intégration entre les différents modules matériels et logiciels.

La prise en compte de ces contraintes est essentielle afin de concevoir une architecture réaliste, adaptée aux ressources disponibles et aux objectifs du projet.

2.4 Environnement logiciel

Visual Studio Code [12] : Visual Studio Code est un éditeur de code source développé par Microsoft. **Nous avons utilisé cet outil pour programmer et organiser les différents modules logiciels du robot AXEL.**

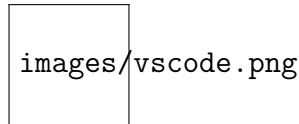


FIGURE 2.2 – Logo de Visual Studio Code

GitHub [13] : GitHub est une plateforme de gestion de code source basée sur le système de versionnement Git. **Nous avons utilisé cette plateforme pour héberger le code du projet et suivre les différentes versions du développement.**

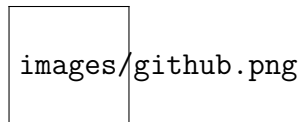


FIGURE 2.3 – Logo de GitHub

ROS2 [14] : ROS2 est un framework open source dédié au développement d'applications robotiques. **Nous avons utilisé cet outil pour gérer la communication entre les différents modules du robot AXEL.**

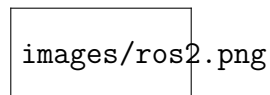


FIGURE 2.4 – Logo de ROS2

Arduino IDE [15] : Arduino IDE est un environnement de développement permettant de programmer le microcontrôleur ESP32. **Nous avons utilisé cet outil pour développer les programmes contrôlant les capteurs et les actionneurs du robot.**

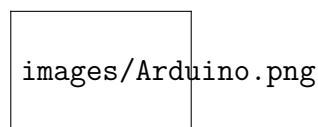


FIGURE 2.5 – Logo de Arduino IDE

Ubuntu [16] : Ubuntu est un système d'exploitation basé sur Linux largement utilisé dans le domaine de la robotique. **Nous avons utilisé ce système comme plateforme principale pour le développement et l'exécution des applications du robot AXEL.**



images/ubuntu1.png

FIGURE 2.6 – Logo de Ubuntu

Python [17] : Python est un langage de programmation largement utilisé dans les domaines de l'intelligence artificielle et de la robotique. **Nous avons utilisé ce langage pour développer plusieurs modules logiciels du robot AXEL.**



images/python1.png

FIGURE 2.7 – Logo de Python

C++ [34] : Les langages C++ sont largement utilisés pour le développement de systèmes embarqués. **Nous avons utilisé ces langages pour programmer certaines parties du système et assurer le contrôle des composants matériels du robot.**



images/c++.png

FIGURE 2.8 – Logo du langage C++

React [36] : React est une bibliothèque JavaScript développée par Meta, utilisée pour la création d'interfaces utilisateur interactives et dynamiques. **React est utilisé pour concevoir le tableau de bord permettant de visualiser et de contrôler le robot en temps réel.**



images/react.png

FIGURE 2.9 – Bibliothèque React

Electron [37] : Electron est un framework permettant de développer des applications desktop multiplateformes en utilisant des technologies web. **il est utilisé pour transformer le tableau de bord en une application bureau autonome.**



images/electron.png

FIGURE 2.10 – Framework Electron

Three.js [38] : Three.js est une bibliothèque JavaScript dédiée à la création de graphiques 3D dans le navigateur. **elle est utilisée pour la visualisation tridimensionnelle du robot et la simulation de ses mouvements.**

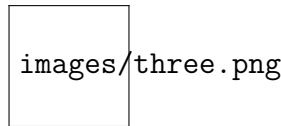


FIGURE 2.11 – Bibliothèque Three.js

TypeScript [39] : TypeScript est un langage de programmation basé sur JavaScript, intégrant un système de typage statique. Il est utilisé dans le projet **comme langage principal pour améliorer la fiabilité, la lisibilité et la maintenabilité du code.**

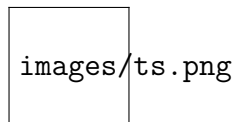


FIGURE 2.12 – Langage TypeScript

draw.io [18] : draw.io est un outil en ligne permettant la création de diagrammes techniques. **Nous avons utilisé cet outil pour concevoir les diagrammes UML et les schémas d'architecture du système.**

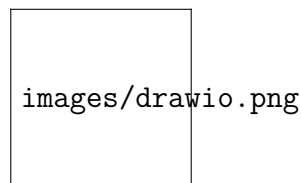


FIGURE 2.13 – Logo de draw.io

Overleaf [19] : Overleaf est une plateforme collaborative basée sur LaTeX permettant la rédaction de documents scientifiques. **Nous avons utilisé cette plateforme pour rédiger et structurer le rapport du projet AXEL.**

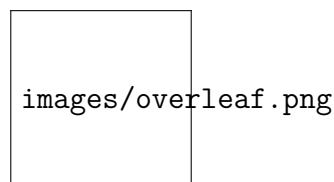


FIGURE 2.14 – Logo de Overleaf

RViz [20] : RViz est un outil de visualisation 3D open source intégré à ROS2. Il permet d'afficher en temps réel l'état du robot, les données capteurs et les transformations

entre repères. Nous l'avons utilisé pour visualiser et déboguer les modules de perception du robot AXEL durant les phases de test.

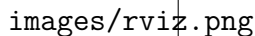
A rectangular box containing the text "images/rviz.png".

FIGURE 2.15 – Logo de RViz

MoveIt [33] : MoveIt est un framework robotique basé sur ROS2 dédié à la planification et au contrôle des mouvements. Il permet de calculer des trajectoires optimales en tenant compte des contraintes cinématiques et des collisions.

Nous avons utilisé MoveIt dans le projet AXEL pour planifier et exécuter les mouvements des bras et des articulations du robot de manière précise et fluide.

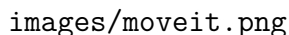
A rectangular box containing the text "images/moveit.png".

FIGURE 2.16 – Logo de MoveIt

2.5 Environnement matériel

Raspberry Pi 5 [21] : Le Raspberry Pi 5 est un ordinateur monocarte haute performance développé par la Raspberry Pi Foundation. Il constitue l'unité centrale du robot AXEL, hébergeant l'environnement ROS2 et orchestrant la communication avec les modules d'intelligence artificielle et les contrôleurs embarqués.

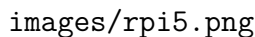
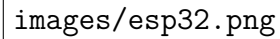
A rectangular box containing the text "images/rpi5.png".

FIGURE 2.17 – Raspberry Pi 5

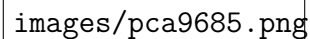
ESP32 [22] : L'ESP32 est un microcontrôleur développé par Espressif Systems, intégrant des capacités WiFi et Bluetooth. Dans le projet AXEL, il est dédié au contrôle de la tête du robot, recevant les commandes du Raspberry Pi 5 via le bus I2C et les transmettant au driver PCA9685.



images/esp32.png

FIGURE 2.18 – Microcontrôleur ESP32

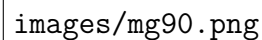
PCA9685 [23] est un driver PWM 16 canaux développé par NXP Semiconductors, communiquant via le protocole I2C. Dans le projet AXEL, ce module est utilisé pour piloter les servomoteurs de la tête ainsi que pour assurer le contrôle des mouvements des bras du robot.



images/pca9685.png

FIGURE 2.19 – Driver PWM PCA9685

MG90 [25] : Le SG90 est un micro-servomoteur léger et compact développé par Tower Pro. Grâce à son faible encombrement et sa précision de positionnement, il est utilisé dans le projet **AXEL** pour assurer les mouvements fins de la tête du robot.



images/mg90.png

FIGURE 2.20 – Servomoteur SG90

MG90S [26] : Le MG90S est une version améliorée du MG90, intégrant des engrenages métalliques offrant une meilleure durabilité et une résistance accrue à l'usure. Il est également utilisé pour les articulations de la tête du robot **AXEL** nécessitant une tenue mécanique plus robuste.

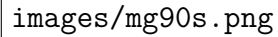
A square box containing the text "images/mg90s.png", serving as a placeholder for the image of the MG90S servo motor.

FIGURE 2.21 – Servomoteur MG90S

MG996R [27] : Le MG996R est un servomoteur haute performance offrant un couple élevé, conçu pour des applications nécessitant davantage de puissance mécanique. Dans le projet **AXEL**, il est dédié au contrôle des articulations des bras du robot.

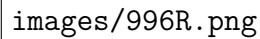
A square box containing the text "images/996R.png", serving as a placeholder for the image of the MG996R servo motor.

FIGURE 2.22 – Servomoteur MG996R

DS3218 [28] : Le servomoteur DS3218 est un moteur numérique offrant un couple élevé de $20 \text{ kg} \cdot \text{cm}$ avec une rotation de 180° . Il est reconnu pour sa robustesse et sa précision, ce qui le rend adapté aux systèmes robotiques nécessitant des mouvements fiables et contrôlés.

Dans le robot **AXEL**, ce servomoteur est principalement utilisé pour les articulations intermédiaires, telles que les coudes ou les poignets, où un bon compromis entre puissance et précision est requis.

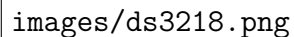
A square box containing the text "images/ds3218.png", serving as a placeholder for the image of the DS3218 servo motor.

FIGURE 2.23 – Servomoteur DS3218 (20 kg , 180°)

DS3225 [29] : Le servomoteur DS3225 est un moteur numérique haute performance offrant un couple élevé pouvant atteindre $25 \text{ kg} \cdot \text{cm}$. Il est conçu pour des applications

robotiques nécessitant une grande précision et une forte capacité de charge.

Dans le projet **AXEL**, ce servomoteur est utilisé pour les articulations critiques nécessitant une puissance importante, telles que les épaules ou les parties principales des bras, garantissant ainsi une stabilité et un mouvement fluide du robot.



FIGURE 2.24 – Servomoteur DS3225 (25 kg, 180°)

Microphone Spektra [30] : Le microphone Spektra est un microphone USB compact utilisé dans le projet **AXEL** pour la capture des commandes vocales de l'utilisateur. Il assure une acquisition audio claire et précise, constituant ainsi le premier maillon de la chaîne de traitement du langage naturel.



FIGURE 2.25 – Microphone Spektra

Haut-parleur USB [31] : Un haut-parleur USB est intégré au robot **AXEL** pour la restitution audio. Il permet au robot de communiquer vocalement avec l'utilisateur en diffusant les réponses générées par les modules d'intelligence artificielle.



FIGURE 2.26 – Haut-parleur USB

Filament Hyper PLA RFID [32] : Le filament Hyper PLA RFID est un matériau utilisé pour l'impression 3D des composants mécaniques du robot. Il se distingue par sa facilité d'impression, sa bonne résistance mécanique et sa stabilité dimensionnelle.

Dans le projet **AXEL**, ce filament est utilisé pour la fabrication des différentes pièces du robot, telles que les supports, les structures des bras et les éléments de fixation. Il permet d'obtenir des pièces légères, précises et suffisamment robustes pour les applications robotiques.



FIGURE 2.27 – Filament Hyper PLA RFID utilisé pour l'impression 3D

2.6 Architecture générale du système

L'architecture générale du robot AXEL s'appuie sur une organisation en couches fonctionnelles, offrant une structure claire et cohérente pour les différents composants du système.

Cette approche facilite une séparation efficace des responsabilités entre les modules matériels et logiciels, tout en améliorant la communication, la maintenance et l'évolution du système.

Elle garantit également une meilleure modularité, permettant l'ajout ou la modification de certains composants sans affecter l'ensemble de l'architecture.

La figure suivante illustre une vue d'ensemble de l'architecture du robot AXEL.



FIGURE 2.28 – Architecture générale du robot AXEL

L'architecture du robot AXEL est organisée en cinq couches fonctionnelles permettant de structurer les différentes composantes du système et d'assurer une interaction efficace entre les modules matériels et logiciels :

- **Couche de perception** : Cette couche constitue le premier niveau du système et regroupe les différents capteurs permettant au robot de percevoir son environnement. Elle inclut notamment la caméra, le microphone et d'autres capteurs utilisés pour collecter des données sur l'environnement. Les informations acquises sont ensuite transmises aux modules de traitement pour être analysées.
- **Couche de traitement embarqué** : Cette couche regroupe les unités matérielles responsables du traitement des données et du contrôle du robot. Elle comprend principalement la plateforme **Raspberry Pi** ainsi que des microcontrôleurs tels que **ESP32** couplé aux drivers PCA9685 , qui assurent la gestion des capteurs, des actionneurs et l'exécution des différentes tâches du système.

- **Couche de communication** : Cette couche permet la communication entre les différents modules du robot et assure l'échange des données entre les composants matériels et logiciels. Dans notre architecture, cette communication est principalement assurée par le middleware **ROS2**.
- **Couche d'intelligence artificielle** : Cette couche est chargée du traitement avancé des données et de la prise de décision. Elle intègre plusieurs techniques d'intelligence artificielle, notamment la **vision par ordinateur (Computer Vision)** et le **traitement du langage naturel (NLP)**, permettant au robot d'analyser son environnement et d'interagir avec l'utilisateur.
- **Couche d'application** : Cette couche représente le niveau d'interaction avec l'utilisateur. Elle comprend les interfaces d'interaction homme-robot, les mécanismes de commande vocale ainsi que les outils de monitoring permettant de superviser et de contrôler le fonctionnement du robot AXEL.

2.7 Étude de l'architecture logicielle

L'architecture logicielle du robot AXEL est conçue selon une approche modulaire permettant de structurer les différents composants logiciels du système. Elle assure la coordination et l'échange d'informations entre les modules de perception, de traitement des données, de prise de décision et de contrôle du robot.

Cette architecture s'appuie principalement sur le framework ROS2, qui permet de gérer la communication et l'interaction entre les différents modules logiciels du système.

2.7.1 Organisation des modules logiciels

L'architecture logicielle du robot AXEL est organisée en plusieurs modules indépendants, chacun responsable d'une fonction spécifique du système. Cette organisation modulaire permet de simplifier le développement, la maintenance et l'évolution du robot.

Les principaux modules logiciels du système sont :

- **Module de perception** : chargé de collecter et d'analyser les données provenant des capteurs tels que la caméra et le microphone.
- **Module de traitement** : responsable du traitement des données et de l'exécution des algorithmes nécessaires au fonctionnement du robot.
- **Module de décision** : analyse les informations disponibles afin de déterminer les actions que le robot doit effectuer.
- **Module de contrôle** : assure le pilotage des actionneurs et le contrôle des mouvements du robot.

2.7.2 Architecture basée sur ROS2

L'architecture logicielle du robot **AXEL** repose sur le framework ROS2, qui permet d'organiser le système sous forme de nœuds logiciels indépendants communiquant entre eux à travers des *topics* et des services. Cette approche facilite la modularité du système et permet d'assurer un échange efficace des données entre les différents composants du robot.

Les principaux nœuds ROS2 du système sont les suivants :

- **Nœud de perception** : responsable de la collecte des données provenant des capteurs, notamment le microphone et le capteur de distance, puis de leur publication sur les topics ROS2 correspondants.
- **Nœud d'intelligence artificielle** : chargé du traitement des commandes vocales à l'aide de l'API Gemini, puis de la transmission des décisions au module de contrôle.
- **Nœud de contrôle moteur** : reçoit les commandes de mouvement et les transmet aux microcontrôleurs (ESP32) et aux modules PCA9685 via le protocole I2C afin de piloter les actionneurs du robot.
- **Nœud de monitoring** : permet de visualiser l'état du robot et les données des capteurs à travers l'outil RViz.



FIGURE 2.29 – Architecture Logicielle du robot AXEL (basee sur ros2)

2.7.3 Justification du choix de ROS2

Par ailleurs, des plateformes robotiques avancées telles que NVIDIA Isaac ont été étudiées. Cette solution propose des outils performants pour la simulation et l'intelligence artificielle, notamment grâce à l'accélération par GPU.

Cependant, elle requiert des ressources matérielles importantes (GPU ou plateformes NVIDIA Jetson), engendrant un coût élevé et une consommation énergétique importante,

peu compatibles avec un système embarqué.

À l'inverse, ROS 2 se distingue par sa flexibilité et sa capacité à fonctionner sur des plateformes légères comme le Raspberry Pi. Son architecture distribuée, basée sur DDS (Data Distribution Service), permet une communication efficace entre les différents modules du robot.

Ainsi, le choix de ROS 2 s'avère plus adapté au projet AXEL, offrant une solution modulaire, légère et compatible avec les contraintes matérielles du système.

Conclusion

Ce chapitre a permis de structurer les fondations du projet **AXEL** à travers une analyse détaillée des besoins fonctionnels et non fonctionnels, ainsi que l'identification des acteurs et des contraintes du système.

La définition des environnements matériel et logiciel, combinée à la conception de l'architecture globale, constitue un élément clé garantissant la cohérence, la modularité et l'évolutivité de la solution proposée. Cette phase de conception joue un rôle déterminant dans la maîtrise de la complexité du système, en assurant une intégration efficace entre les différents modules du robot.

Le chapitre suivant sera consacré à la mise en œuvre pratique de cette architecture, en détaillant le développement et l'intégration des différents composants du système AXEL.

3

Chapitre

Intégration matérielle et tests expérimentaux

Introduction

Ce chapitre est consacré à la première phase concrète du développement de la plateforme **AXEL**, correspondant au Sprint 1 du projet. Il décrit l'ensemble des étapes de configuration et d'installation nécessaires à l'établissement d'un environnement de développement fonctionnel et cohérent.

Cette phase représente le socle technique sur lequel reposent tous les développements ultérieurs. Elle englobe l'installation du système d'exploitation, la configuration du middleware robotique ROS2, la mise en place du microcontrôleur ESP32, ainsi que le câblage et les tests de communication entre les différents composants matériels du robot.

Ce chapitre ne se limite pas à la configuration de l'environnement, mais présente également l'intégration mécanique, électronique et embarquée du robot AXEL, ainsi que la validation expérimentale du système.

3.1 Organisation du sprint

3.1.1 Backlog du Sprint 1

Le tableau suivant présente l'ensemble des tâches réalisées dans le cadre du Sprint 1, avec leur durée estimée.

TABLE 3.1 – Backlog du Sprint 1 — Intégration matérielle et tests expérimentaux

ID	Sprint	User Story	Durée
1	Intégration matérielle et tests expérimentaux	En tant que développeur, je souhaite installer et configurer Ubuntu 24.04 LTS sur le Raspberry Pi 5 afin de disposer d'un système d'exploitation stable et compatible avec ROS2.	2 jours
2		En tant que développeur, je souhaite installer et configurer ROS2 Jazzy Jalisco afin d'assurer la communication entre les différents modules logiciels du robot AXEL.	3 jours
3		En tant que développeur, je souhaite configurer l'ESP32 via Arduino IDE afin de programmer le contrôle des actionneurs du robot.	2 jours
4		En tant que développeur, je souhaite câbler les modules PCA9685 et les servomoteurs MG90, MG90S et MG996R afin d'assurer le contrôle matériel des mouvements du robot.	2 jours
5		En tant que développeur, je souhaite tester et valider la communication I2C entre le Raspberry Pi 5 et l'ESP32 afin de garantir la fiabilité des échanges de données.	2 jours
6		En tant que développeur, je souhaite initialiser le dépôt GitHub du projet AXEL afin d'assurer la gestion et le suivi des versions du code source.	1 jour

3.2 Installation du système d'exploitation

3.2.1 Choix d'Ubuntu 24.04 LTS

Le choix du système d'exploitation représente une décision fondamentale dans l'architecture logicielle d'un système robotique. Pour le projet **AXEL**, nous avons retenu **Ubuntu 24.04 LTS (Noble Numbat)**, installé sur le Raspberry Pi 5 en tant que plateforme principale d'exécution.

Ce choix se justifie par plusieurs raisons techniques :

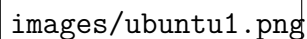
- **Compatibilité native avec ROS2 Jazzy** : Ubuntu 24.04 LTS est la distribution officiellement supportée par ROS2 Jazzy Jalisco, garantissant ainsi une intégration optimale et sans conflits de dépendances.
- **Support à long terme** : en tant que version LTS, Ubuntu 24.04 bénéficie d'un support de cinq ans, assurant la stabilité et la maintenance du système tout au long du cycle de vie du projet.

- **Écosystème riche** : la disponibilité d’une large bibliothèque de paquets robotiques, d’outils de développement et de pilotes matériels facilite considérablement l’intégration des composants du robot.
- **Performances optimisées** : la version 64 bits d’Ubuntu 24.04 exploite pleinement les capacités du processeur ARM Cortex-A76 du Raspberry Pi 5, offrant des performances supérieures aux versions précédentes du système.

3.2.2 Procédure d’installation

L’installation d’Ubuntu 24.04 LTS sur le Raspberry Pi 5 a été réalisée à l’aide de l’outil **Raspberry Pi Imager**, qui permet de flasher l’image du système d’exploitation directement sur une carte microSD. Les étapes suivantes ont été effectuées :

1. Téléchargement de l’image officielle Ubuntu 24.04 LTS pour Raspberry Pi 5.
2. Flashage de l’image sur une carte microSD de 64 Go via Raspberry Pi Imager.
3. Configuration initiale du système : nom d’hôte, utilisateur, mot de passe et activation du SSH.
4. Mise à jour complète du système via les commandes `apt update` et `apt upgrade`.
5. Configuration du réseau Wi-Fi pour assurer la connectivité du Raspberry Pi 5.



images/ubuntu1.png

FIGURE 3.1 – Système d’exploitation Ubuntu 24.04 LTS

3.3 Installation et configuration de ROS2 Jazzy

3.3.1 Présentation de ROS2 Jazzy Jalisco

ROS2 Jazzy Jalisco est la version LTS du framework robotique ROS2, publiée en mai 2024 et maintenue jusqu’en mai 2029. Elle représente le middleware central du projet **AXEL**, assurant la communication entre les différents modules logiciels du robot.

ROS2 Jazzy offre plusieurs avantages déterminants pour notre architecture :

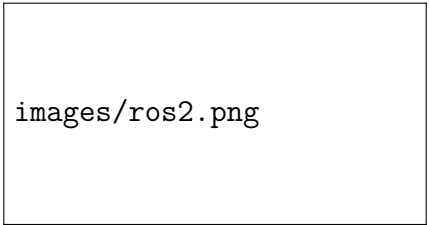
- **Architecture orientée nœuds** : chaque module fonctionnel du robot est encapsulé dans un nœud ROS2 indépendant, favorisant la modularité et la réutilisabilité du code.

- **Communication temps réel** : le middleware DDS (Data Distribution Service) utilisé par ROS2 garantit des échanges de données fiables et à faible latence entre les composants.
- **Support natif d'Ubuntu 24.04** : la compatibilité native élimine les problèmes de dépendances et simplifie le processus d'installation.
- **Écosystème riche** : la disponibilité de packages tels que MoveIt 2, RViz et Nav2 permet d'accélérer le développement des modules de planification et de visualisation.

3.3.2 Procédure d'installation de ROS2 Jazzy

L'installation de ROS2 Jazzy sur Ubuntu 24.04 LTS a été réalisée en suivant la procédure officielle de la documentation ROS2. Les principales étapes sont les suivantes :

1. Configuration des sources APT et ajout de la clé GPG officielle de ROS2.
2. Installation de ROS2 Jazzy Desktop incluant les outils de visualisation RViz.
3. Configuration des variables d'environnement via le fichier `.bashrc`.
4. Installation des dépendances nécessaires via `rosdep`.
5. Création et configuration du workspace ROS2 du projet AXEL.
6. Validation de l'installation via les outils de test intégrés.



images/ros2.png

FIGURE 3.2 – Framework ROS2 Jazzy Jalisco

3.4 Configuration de l'ESP32

3.4.1 Rôle de l'ESP32 dans le système AXEL

Au sein de l'architecture du robot **AXEL**, le microcontrôleur **ESP32** joue un rôle central dans le contrôle des actionneurs. Il reçoit les commandes de mouvement transmises par le Raspberry Pi 5 via le bus I2C et les transmet aux modules PCA9685, qui pilotent en temps réel les servomoteurs de la tête et des bras du robot.

3.4.2 Configuration via Arduino IDE

La programmation de l'ESP32 a été réalisée à l'aide de l'**Arduino IDE**, configuré avec l'extension **ESP32 Board Manager** développée par Espressif Systems. Cette extension permet de compiler et de téléverser du code C/C++ directement sur le microcontrôleur via une connexion USB.

Les étapes de configuration suivantes ont été effectuées :

1. Installation de l'Arduino IDE sur le poste de développement.
2. Ajout de l'URL du gestionnaire de cartes ESP32 dans les préférences de l'Arduino IDE.
3. Installation du package **ESP32 Board Manager** par Espressif Systems.
4. Sélection de la carte **ESP32 Dev Module** dans le gestionnaire.
5. Configuration du port série et de la vitesse de communication.
6. Validation par un programme de test de communication I2C.

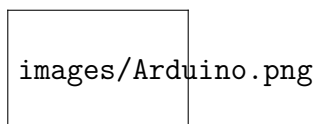


FIGURE 3.3 – Environnement Arduino IDE pour la programmation de l'ESP32

3.5 Conception mécanique et modélisation 3D de la tête

3.5.1 Architecture mécanique de la tête

La tête du robot AXEL représente un sous-système mécatronique essentiel, conçu pour assurer une interaction expressive et naturelle avec l'utilisateur. Elle a été conçue de manière modulaire afin de permettre une intégration efficace des actionneurs et une reproduction réaliste des mouvements humains.

Sur le plan mécanique, la tête repose sur un ensemble de liaisons pivot, chacune associée à un servomoteur, permettant de générer plusieurs degrés de liberté. Les principaux mouvements assurés sont :

- la rotation horizontale de la tête (gauche/droite),
- l'inclinaison verticale (haut/bas),
- le mouvement de la mâchoire (ouverture/fermeture),
- les mouvements expressifs du visage (yeux, paupières, sourcils).

Chaque mouvement est produit par un servomoteur, mais il n'est pas appliqué directement aux parties visibles. Au lieu de cela, il est transmis via une structure mécanique constituée de supports, d'axes de rotation et, parfois, de mécanismes à levier. Ce principe de transmission permet :

- d'adapter le mouvement du servomoteur aux contraintes physiques,
- d'améliorer la fluidité des mouvements,
- de réduire les efforts mécaniques appliqués aux composants.

Ainsi, la tête du robot AXEL ne se résume pas à un simple assemblage de servomoteurs, mais constitue un système mécanique structuré, capable de convertir des commandes numériques en mouvements physiques cohérents.

3.5.2 Modèle 3D et intégration des servomoteurs

La conception de la tête a été réalisée à partir d'un modèle 3D imprimé en PLA, offrant une structure à la fois légère, robuste et facilement modifiable. Ce modèle 3D intègre des logements spécifiques pour chaque servomoteur, assurant un positionnement précis et un alignement optimal avec les axes de rotation des différentes parties mobiles.

Les principaux éléments mécaniques intégrés dans le modèle sont :

- le support des yeux et du mécanisme de regard,
- le système des paupières,
- la structure de fixation des sourcils,
- le mécanisme de la mâchoire,
- le support de la bouche,
- l'axe de rotation principal de la tête,
- les systèmes de fixation (visserie et supports).

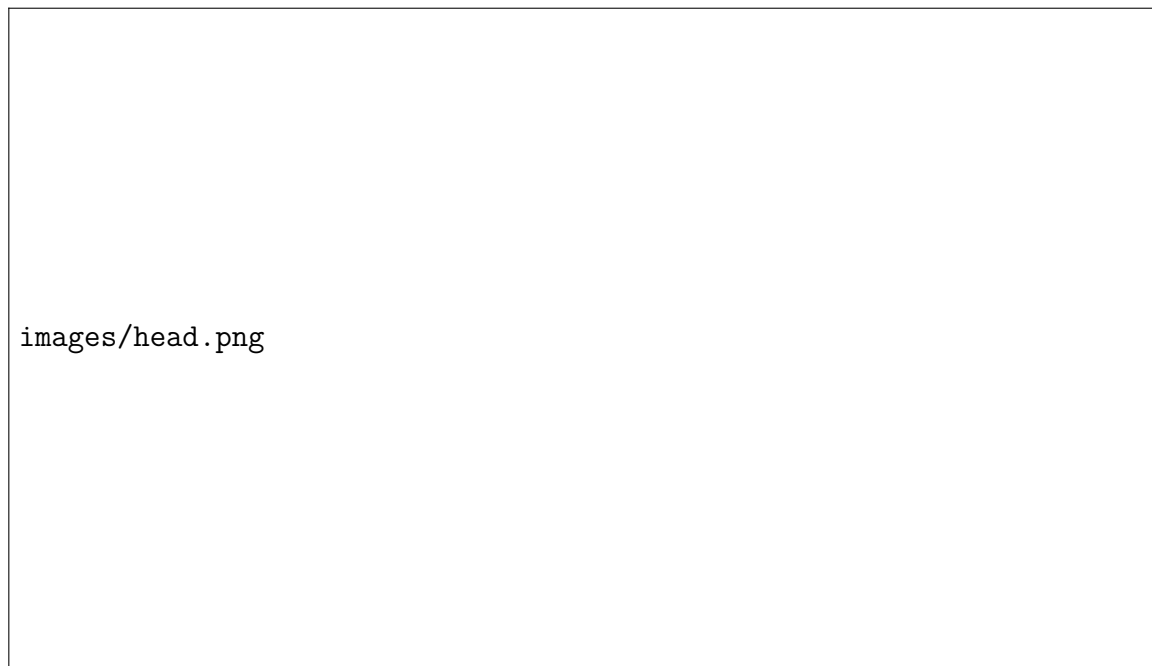


FIGURE 3.4 – Modèle 3D détaillé de la tête du robot AXEL

Dans l'ensemble de la tête, les mouvements des servomoteurs ne sont pas appliqués directement aux éléments visibles, mais sont transmis au travers de mécanismes mécaniques adaptés à chaque fonction.

Par exemple, pour la mâchoire, le mouvement de rotation du servomoteur est transformé en ouverture et fermeture grâce à un système de transmission, tel qu'un levier ou une liaison mécanique.

De manière similaire :

- les mouvements des yeux sont réalisés autour d'axes de rotation précis permettant l'orientation du regard,
- les paupières utilisent des mécanismes de pivot pour simuler le clignement,
- les sourcils sont actionnés pour reproduire des expressions faciales,
- la rotation de la tête repose sur un axe principal permettant l'orientation globale.

Chaque sous-système mécanique est ainsi conçu pour transformer la rotation du servomoteur en un mouvement fonctionnel adapté à l'élément concerné.

Cette conception modulaire facilite :

- l'intégration des composants électroniques,
- la maintenance et le remplacement des servomoteurs,
- l'évolution future du système.

Enfin, la géométrie du modèle 3D joue un rôle déterminant dans la définition des limites mécaniques du système. Elle conditionne les plages de fonctionnement des servomoteurs,

en évitant les collisions entre les pièces et en garantissant la protection de l'ensemble de la structure.

3.6 Câblage et intégration matérielle

3.6.1 Architecture de câblage

L'intégration matérielle du robot **AXEL** repose sur une architecture de câblage hiérarchique organisée autour du bus I2C. Le Raspberry Pi 5 représente le maître du bus, tandis que l'ESP32 et les modules PCA9685 en sont les périphériques esclaves.

3.6.2 Connexion des modules PCA9685

Les modules **PCA9685** ont été connectés au bus I2C du Raspberry Pi 5 selon le schéma suivant :

- **VCC** : alimentation 3.3V depuis le Raspberry Pi 5.
- **GND** : masse commune.
- **SDA** : ligne de données I2C (GPIO 2 du Raspberry Pi 5).
- **SCL** : ligne d'horloge I2C (GPIO 3 du Raspberry Pi 5).
- **V+** : alimentation 5V pour les servomoteurs.

3.6.3 Connexion des servomoteurs

Les servomoteurs ont été répartis sur les modules PCA9685 selon leur rôle fonctionnel :

- **Module PCA9685 #1 — Tête du robot** : accueille les servomoteurs **MG90** et **MG90S** responsables des mouvements de rotation, d'inclinaison et d'expression faciale de la tête.
- **Module PCA9685 #2 — Bras du robot** : accueille les servomoteurs **MG996R** dédiés au contrôle des articulations des deux bras du robot.

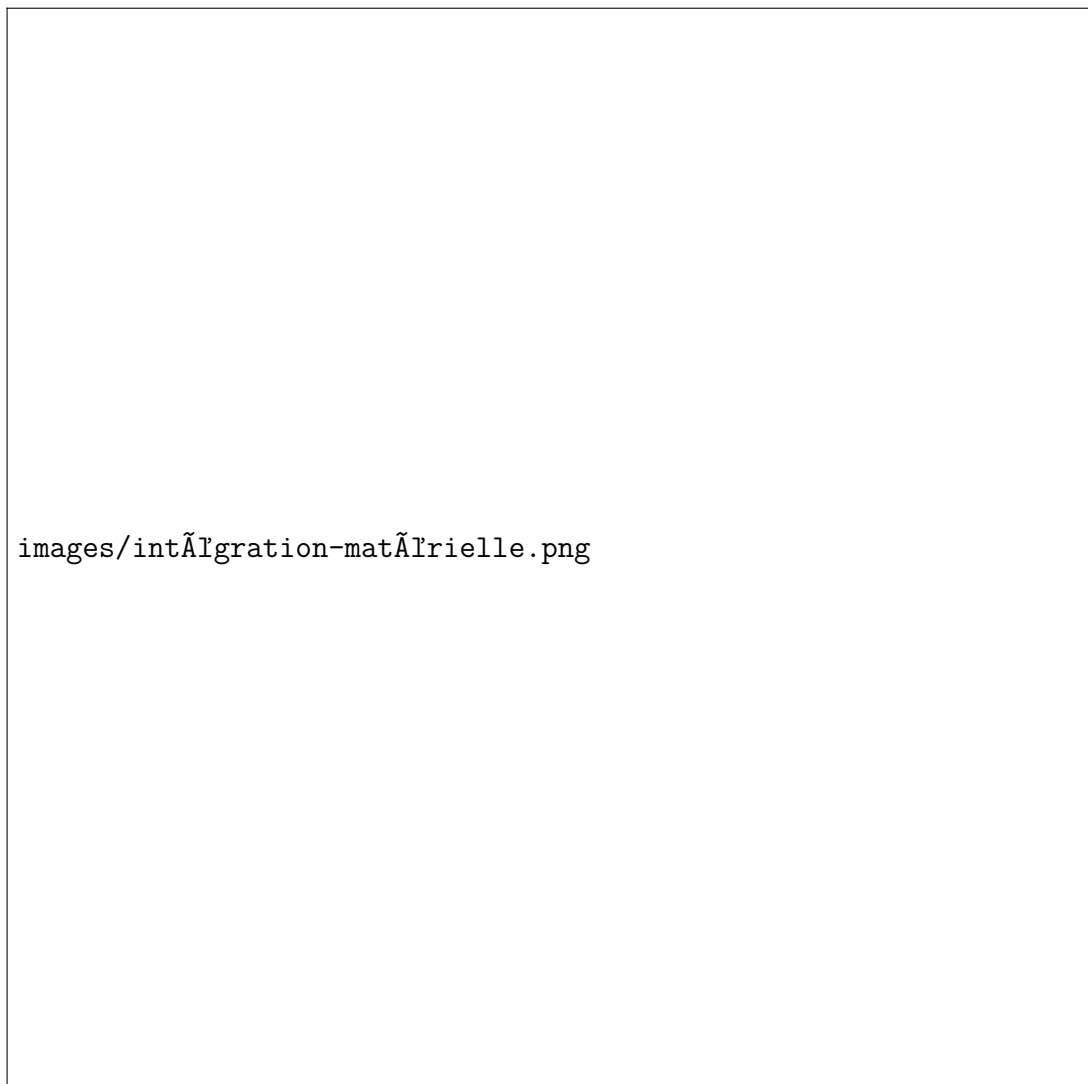


FIGURE 3.5 – Architecture matérielle de la tête du robot AXEL

La figure 3.5 présente l’architecture matérielle de la tête du robot AXEL. Le Raspberry Pi représente l’unité centrale de traitement du système, assurant la gestion des modules audio via les interfaces USB. Le microphone est utilisé pour la capture des commandes vocales, tandis que le haut-parleur permet la restitution des réponses audio.

Le microcontrôleur ESP32 est dédié au contrôle des actionneurs. Il communique avec le Raspberry Pi à travers une liaison UART bidirectionnelle, garantissant un échange fiable et en temps réel des données et des commandes.

Le pilotage des servomoteurs est assuré par le module PCA9685, connecté via le bus I2C. Ce module génère des signaux PWM permettant un contrôle précis des mouvements de la tête du robot. Une alimentation externe est utilisée afin d’assurer la stabilité du système et de fournir la puissance nécessaire au bon fonctionnement des servomoteurs.

L’architecture globale du système peut être résumée comme suit :

Raspberry Pi (traitement) → ESP32 (contrôle temps réel) → PCA9685 (génération PWM) → Servomoteurs → Mouvement mécanique

Cette architecture matérielle constitue la base du contrôle des mouvements du robot, permettant de transformer les commandes numériques en actions mécaniques réelles.

3.7 Contrôle des mouvements de la tête

3.7.1 Mécanisme de rotation de la tête

Le mouvement de rotation de la tête du robot AXEL est assuré par un servomoteur principal positionné au niveau du cou, permettant une orientation horizontale gauche/droite.

Ce mouvement repose sur un axe de rotation mécanique (liaison pivot) assurant la transmission du couple du servomoteur vers la structure de la tête. L'orientation est définie par des angles de consigne envoyés depuis l'ESP32 via le module PCA9685.

Les positions principales utilisées sont :

- 40° : orientation vers la gauche,
- 90° : position centrale,
- 160° : orientation vers la droite.

L'amplitude totale de rotation est donc de 120°, définie en fonction des contraintes mécaniques du modèle 3D. Une rotation complète de 180° n'est pas utilisée afin d'éviter les collisions avec la structure du cou et les contraintes excessives sur le servomoteur.

Afin d'améliorer le réalisme et de réduire les chocs mécaniques, le mouvement de la tête est effectué de manière progressive (interpolation angulaire), permettant un déplacement fluide entre les différentes positions.

Ce mécanisme permet au robot d'effectuer un balayage de son environnement, renforçant ainsi ses capacités de perception et d'interaction avec l'utilisateur.

3.7.2 Synchronisation labiale par RMS

Afin de rendre l'interaction vocale du robot AXEL plus naturelle et réaliste, un mécanisme de synchronisation labiale basé sur le Root Mean Square (RMS) a été implémenté.

Le RMS représente une méthode robuste permettant d'évaluer l'énergie instantanée d'un signal audio, en fournissant une estimation fiable de son intensité sonore.

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2}$$

où :

- x_i représente les échantillons audio,

— N le nombre total d'échantillons.

Cette valeur permet d'estimer l'intensité sonore du signal à un instant donné.

Le principe de fonctionnement repose sur une analyse en temps réel du flux audio. Celui-ci est découpé en blocs temporels de courte durée. Pour chaque bloc, les étapes suivantes sont réalisées :

1. acquisition des échantillons audio,
2. calcul de la valeur RMS,
3. conversion de cette valeur en angle de rotation,
4. commande du servomoteur de la mâchoire.

La chaîne de traitement peut être résumée comme suit :

$$\text{Audio} \rightarrow \text{RMS} \rightarrow \text{Angle} \rightarrow \text{Servomoteur (mâchoire)}$$

La conversion de la valeur RMS en angle est effectuée selon la relation suivante :

$$\theta = \theta_{min} + RMS \times (\theta_{max} - \theta_{min})$$

avec :

- $\theta_{min} = 70^\circ$ correspondant à la bouche fermée,
- $\theta_{max} = 180^\circ$ correspondant à la bouche ouverte.

Ainsi, une augmentation de l'intensité sonore entraîne une ouverture proportionnelle de la bouche, tandis qu'un signal faible ou nul maintient la mâchoire en position fermée.

Ce mécanisme ne se limite pas à un traitement logiciel : l'angle calculé est appliqué à un servomoteur qui actionne mécaniquement la mâchoire via un système de transmission. Le signal audio est ainsi converti en mouvement physique réel.

Cette approche permet d'obtenir une synchronisation dynamique entre la parole et le mouvement de la bouche, améliorant significativement le réalisme de l'interaction homme-robot.

3.8 Sécurité et protection du système

3.8.1 Limites mécaniques des servomoteurs

Afin de garantir la fiabilité et la durabilité du système, des limites angulaires minimales et maximales ont été définies pour chaque servomoteur de la tête du robot AXEL.

Ces limites ne sont pas arbitraires, mais résultent directement des contraintes géométriques du modèle 3D et des limites physiques du système.

Le contrôle des angles est réalisé au niveau logiciel à l'aide d'une contrainte de type :

```
angle = constrain(angle, servoMinLimit[ch], servoMaxLimit[ch]);
```

Cette approche assure que chaque servomoteur fonctionne uniquement dans une plage sécurisée.

Les principales protections assurées sont :

- la prévention des collisions entre les pièces mécaniques,
- l'évitement du blocage des articulations,
- la protection des engrenages internes des servomoteurs,
- la réduction des contraintes mécaniques excessives.

Ainsi, les limites logicielles sont directement liées aux contraintes physiques du système, assurant une cohérence entre le contrôle et la structure mécanique.

3.8.2 Gestion énergétique et protection thermique

Les servomoteurs consomment de l'énergie même lorsqu'ils maintiennent une position, ce qui peut entraîner un échauffement et une usure prématurée.

Afin de limiter ces effets, une stratégie de gestion énergétique a été mise en place. Après chaque mouvement, le signal PWM est désactivé à l'aide de la fonction :

```
releaseServo(ch);
```

Cette approche permet :

- de réduire la consommation énergétique,
- de limiter l'échauffement des servomoteurs,
- d'augmenter la durée de vie des composants.

Cependant, cette désactivation implique un relâchement du maintien actif de la position, ce qui nécessite un compromis entre stabilité mécanique et efficacité énergétique.

3.8.3 Analyse des limitations et perspectives d'amélioration

Les tests expérimentaux ont révélé certaines limites dans le comportement des servomoteurs, notamment des écarts entre les angles de consigne et les positions réellement atteintes.

Ces écarts peuvent être expliqués par plusieurs facteurs :

- les tolérances mécaniques des pièces imprimées,
- les variations de tension d'alimentation,
- les caractéristiques internes des servomoteurs (jeu mécanique, précision limitée),
- les délais de communication entre les différents modules.

Ces observations mettent en évidence la nécessité d'améliorations futures, telles que :

- la calibration individuelle des servomoteurs,
- l'utilisation de servomoteurs de meilleure précision,
- l'optimisation de l'alimentation électrique,
- l'intégration de capteurs de retour (feedback) pour un contrôle plus précis.

Malgré ces limitations, les mécanismes de sécurité mis en place permettent d'assurer un fonctionnement global stable et sécurisé du système.

3.9 Tests de communication I2C

3.9.1 Validation de la communication

Afin de valider l'ensemble de la chaîne de communication, des tests fonctionnels ont été conduits à chaque niveau de l'architecture. Ces tests ont permis de vérifier la fiabilité des échanges entre le Raspberry Pi 5, l'ESP32 et les modules PCA9685.

Les tests réalisés comprennent :

- **Test de détection I2C** : vérification de la présence et de l'adressage correct des modules PCA9685 sur le bus I2C via la commande `i2cdetect`.
- **Test de communication série** : validation de l'échange de données entre le Raspberry Pi 5 et l'ESP32 via le protocole UART.
- **Test de contrôle des servomoteurs** : envoi de commandes PWM aux servomoteurs pour vérifier leur réponse et leur positionnement.
- **Test d'intégration ROS2** : publication de messages sur les topics ROS2 et vérification de leur réception par les nœuds de contrôle.

3.9.2 Résultats des tests

Les tests expérimentaux réalisés sur l'architecture matérielle du robot **AXEL** ont permis d'évaluer avec précision le comportement des différents composants, en particulier les servomoteurs utilisés pour le contrôle de la tête.

Au total, **15 servomoteurs** ont été intégrés et testés dans le cadre de ce sprint. Les résultats obtenus montrent que :

- **13 servomoteurs sur 15** présentent un fonctionnement stable et précis, soit un taux de réussite de **86,6%**.
- Ces servomoteurs répondent correctement aux signaux PWM générés par le module PCA9685, avec un positionnement fiable et sans dérive notable.

En revanche, **2 servomoteurs** présentent des anomalies de fonctionnement, notamment :

- des mouvements instables ou saccadés,
- une absence de réponse à certaines commandes,
- une précision insuffisante dans le positionnement,
- ainsi qu'un comportement irrégulier nécessitant parfois l'envoi de la **même commande deux fois** pour obtenir une réponse effective du servomoteur.

De plus, il a été observé que les angles réellement atteints par ces servomoteurs ne correspondent pas toujours avec précision aux valeurs de consigne envoyées. Cette différence entre la position demandée et la position réelle entraîne une perte de fiabilité dans le contrôle des mouvements.

Ces dysfonctionnements peuvent être liés à plusieurs facteurs, notamment :

- des défauts matériels internes des servomoteurs,
- une alimentation insuffisante ou instable,
- des perturbations dans le signal PWM généré par le module PCA9685,
- ou encore des délais de communication et de synchronisation entre le Raspberry Pi, l'ESP32 et les modules de contrôle.

Malgré ces limitations, les résultats obtenus restent globalement satisfaisants pour cette phase initiale, et permettent de valider la faisabilité du contrôle multi-servomoteurs dans l'architecture proposée.

Des améliorations seront apportées lors des sprints suivants afin d'optimiser la fiabilité globale du système et de corriger les problèmes identifiés.

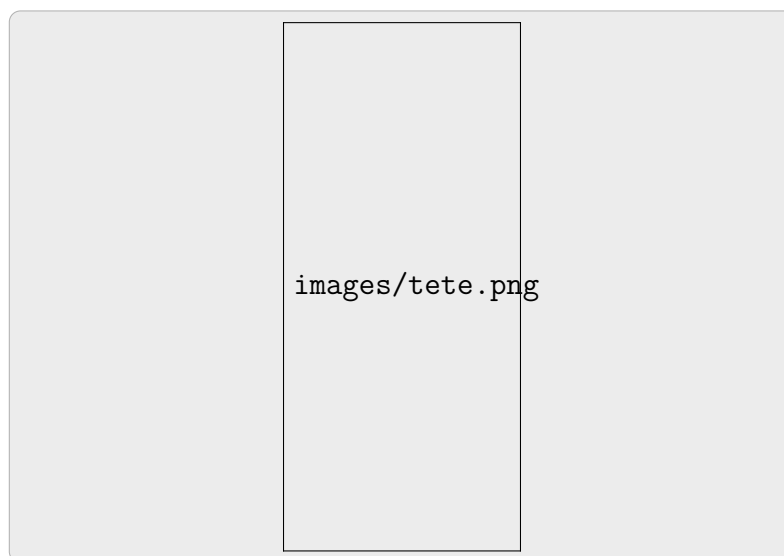


FIGURE 3.6 – Prototype de la tête du robot AXEL après intégration des servomoteurs

Conclusion

Ce chapitre a permis de valider la première intégration complète du système robotique AXEL, en combinant la structure mécanique, le contrôle embarqué et les composants électroniques.

Les résultats expérimentaux obtenus, notamment le contrôle simultané de plusieurs servomoteurs et l'analyse des performances, confirment la faisabilité et la cohérence de l'architecture proposée.

Ces travaux représentent une base solide pour les développements futurs, en particulier l'intégration des modules d'interaction avancée et des fonctionnalités intelligentes du robot.

4

Chapitre

Interaction vocale et traitement du langage naturel

Introduction

Ce chapitre présente l'architecture du module audio et du système de traitement du langage naturel du robot **AXEL**. L'objectif central est de permettre une interaction vocale naturelle et réactive entre l'utilisateur et le robot, en garantissant une latence minimale et une robustesse élevée en conditions réelles.

Le cœur de cette architecture repose sur la **Gemini Live API** de Google DeepMind, qui offre des capacités avancées de traitement audio natif en temps réel, incluant la Détection de l'activité vocale (VAD), la transcription bidirectionnelle et la génération native de réponses audio. Contrairement aux architectures classiques fondées sur une chaîne séquentielle ASR → NLU → TTS, cette approche adopte un pipeline audio natif de bout en bout, réduisant considérablement la latence perçue et améliorant la fluidité globale de l'interaction homme-robot.

4.1 Backlog du Sprint 2

Le tableau suivant présente le backlog détaillé du Sprint 2, décrivant les différentes tâches réalisées pour implémenter le système d'interaction vocale du robot AXEL.

TABLE 4.1 – Backlog du Sprint 2 — Interaction vocale

ID	User Story	Durée
1	En tant que développeur, je souhaite intégrer le microphone Spektra USB afin de permettre la capture des commandes vocales de l'utilisateur en temps réel.	2 jours
2	En tant que développeur, je souhaite développer le nœud de perception ROS2 afin de publier les données audio capturées sur le topic correspondant.	2 jours
3	En tant que développeur, je souhaite connecter et configurer l'API Gemini afin de traiter les commandes vocales et générer des réponses intelligentes adaptées au contexte.	3 jours
4	En tant que développeur, je souhaite intégrer le haut-parleur USB afin de permettre la restitution audio des réponses générées par les modules d'intelligence artificielle.	2 jours
5	En tant que développeur, je souhaite tester et valider le pipeline complet de reconnaissance et de synthèse vocale afin de garantir une interaction fluide et naturelle avec le robot.	2 jours

Afin de mieux comprendre l'intégration de la couche audio et intelligence artificielle dans le système AXEL, il est nécessaire de visualiser son positionnement au sein de l'architecture globale.

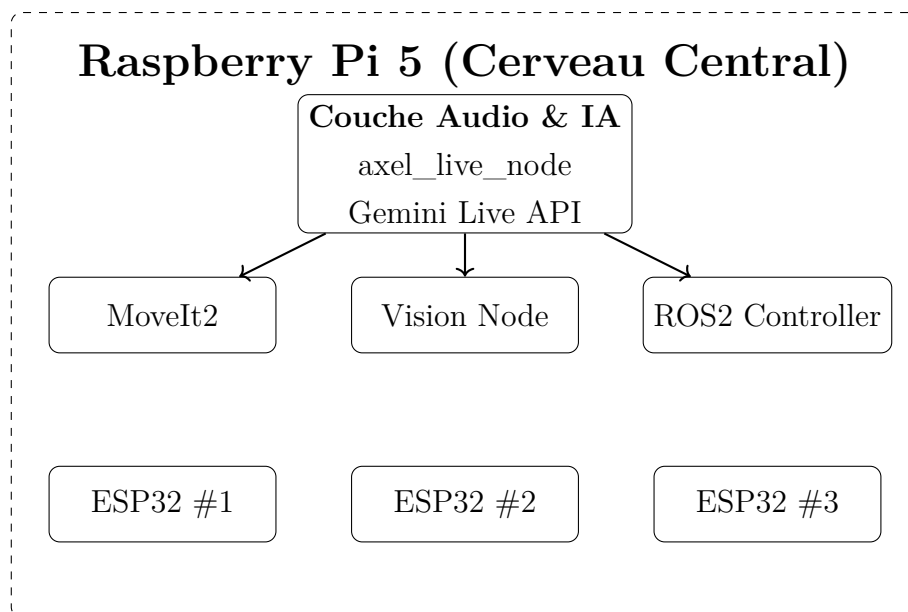


FIGURE 4.1 – Positionnement de la couche audio dans l'architecture globale du robot AXEL

On observe que la couche audio agit comme une interface intelligente entre l'utilisateur et les différents modules du robot.

4.2 Architecture du module vocal

4.2.1 Paramétrage du VAD serveur

Le VAD (Voice Activity Detection) serveur de la **Gemini Live API** est configuré via le paramètre `automatic_activity_detection` dans la configuration `LiveConnectConfig`.

Les paramètres critiques utilisés sont présentés dans le tableau suivant :

TABLE 4.2 – Paramètres du VAD serveur et impact sur la latence

Paramètre	Valeur	Impact
<code>start_of_speech_sensitivity</code>	<code>START_SENSITIVITY_HIGH</code>	Détection rapide du début de parole
<code>end_of_speech_sensitivity</code>	<code>END_SENSITIVITY_HIGH</code>	Fin de parole détectée rapidement
<code>silence_duration_ms</code>	100 ms	Délai avant validation de fin de parole
<code>prefix_padding_ms</code>	20 ms	Ajout de contexte audio avant la parole
<code>activity_handling</code>	<code>START_OF_ACTIVITY_INTERRUPTS</code>	Permet l'interruption naturelle de la conversation

Optimisation : Le paramètre `END_SENSITIVITY_HIGH` est le plus impactant sur la latence. Une configuration plus faible (`END_SENSITIVITY_LOW`) entraîne un délai supplémentaire de 500 ms à 2 secondes avant la détection de la fin de parole.

4.2.2 Choix du Voice Activity Detection (VAD)

Le choix du Voice Activity Detection (VAD) dans le système AXEL repose sur des contraintes de latence, de robustesse et d'efficacité du traitement audio en temps réel.

Plusieurs approches ont été étudiées pour le traitement du signal audio. Une première solution consiste à transmettre en continu le flux audio vers le module d'intelligence artificielle. Toutefois, cette méthode entraîne un traitement permanent, augmentant la latence et la consommation des ressources, tout en étant très sensible au bruit ambiant.

Des méthodes plus simples, comme le seuillage d'énergie (energy threshold), ont également été envisagées. Bien que faciles à implémenter, ces techniques montrent leurs limites en environnement réel, notamment dans la distinction entre la parole et le bruit de fond, ce qui peut provoquer des déclenchements erronés.

À l'inverse, le VAD intégré dans la Gemini Live API repose sur une analyse plus avancée du signal audio, permettant une détection précise et fiable des segments de parole, même en présence de bruit.

Cette approche présente plusieurs avantages :

- réduction de la latence en déclenchant le traitement uniquement lors de la parole,
- meilleure robustesse face au bruit ambiant,
- amélioration de la précision de la détection vocale,
- optimisation de la consommation des ressources.

Ainsi, le choix du VAD s'impose comme une solution adaptée aux exigences du projet AXEL, garantissant une interaction vocale fluide, rapide et fiable dans un environnement réel.

4.2.3 Vue d'ensemble du pipeline audio

Le module vocal du robot **AXEL** repose sur une architecture modulaire organisée en une chaîne de traitement complète, allant de la capture du signal audio jusqu'à la restitution vocale des réponses générées.

Le processus débute par l'acquisition du signal audio via le microphone, suivie de son stockage temporaire dans un buffer afin de réguler le flux de données et de limiter les effets de latence. Un mécanisme de détection de mot-clé (*wake word*) permet ensuite d'activer le système uniquement en présence d'une interaction utilisateur.

Le signal audio est ensuite analysé par un module de détection d'activité vocale (VAD), permettant d'identifier les segments pertinents et de filtrer les silences avant leur transmission au module d'intelligence artificielle.

L'API **Gemini** assure alors le traitement du langage naturel et la génération de réponses adaptées au contexte.

Les résultats produits sont ensuite pris en charge par un module de décision et de distribution des commandes, permettant l'interaction avec les autres composants du robot via ROS2.

Enfin, la réponse générée est convertie en signal audio, stockée dans une file d'attente de sortie, puis diffusée via le haut-parleur, avec une synchronisation éventuelle des mouvements du robot (*lip sync*).

Cette architecture permet d'assurer une interaction vocale fluide, réactive et cohérente avec l'utilisateur.

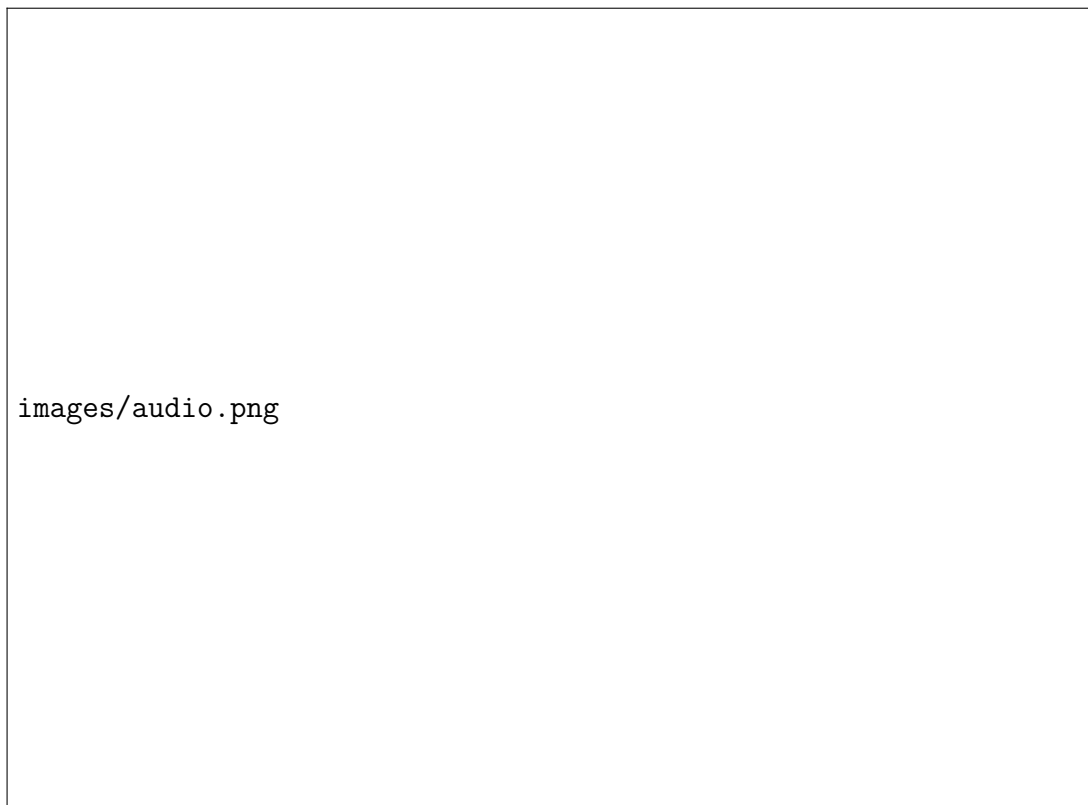


FIGURE 4.2 – Architecture du pipeline d’interaction vocale du robot AXEL

Comme illustré dans la figure 4.2, le système d’interaction vocale s’appuie sur une chaîne de traitement continue allant de la capture du signal audio jusqu’à la génération et la restitution de la réponse. Cette organisation permet d’optimiser la latence et d’assurer une communication réactive et cohérente avec l’utilisateur.

4.2.4 Fonctionnement global du pipeline vocal

Le système d’interaction vocale du robot AXEL repose sur une chaîne de traitement séquentielle permettant de transformer une commande vocale en action ou réponse intelligente. Le fonctionnement global peut être décrit comme suit :

1. **Capture audio** : Le microphone Spektra capte le signal vocal de l’utilisateur en temps réel et le transmet au système.
2. **Détection d’activité vocale (VAD)** : Le module VAD analyse le flux audio afin d’identifier les segments de parole et d’éliminer les silences ou bruits inutiles.
3. **Transcription** : Le signal audio détecté est converti en texte exploitable par le système.
4. **Traitement du langage naturel (NLP)** : Le texte est envoyé à l’API Gemini qui interprète l’intention de l’utilisateur et génère une réponse adaptée au contexte.
5. **Décision** : Le système analyse la réponse générée afin de déterminer s’il s’agit d’une commande d’action (contrôle du robot) ou d’une réponse conversationnelle.

6. **Synthèse vocale** : La réponse finale est convertie en signal audio et diffusée via le haut-parleur pour interagir avec l'utilisateur.

Cette organisation permet d'assurer une interaction fluide, réactive et cohérente entre l'utilisateur et le robot, tout en optimisant la latence globale du système.

4.2.5 Description des composants

Le système d'interaction vocale du robot **AXEL** repose sur plusieurs composants interconnectés, organisés en une chaîne de traitement continue allant de la capture audio à la restitution vocale.

- **Microphone Spektra** : assure la capture du signal audio émis par l'utilisateur avec une qualité adaptée au traitement en temps réel.
- **Nœud de perception ROS2** : acquiert le flux audio en continu, le pré-traite et le publie sur le topic `/audio/input` pour les étapes suivantes du pipeline.
- **Module VAD** : permet de détecter les segments de parole en filtrant les silences et les bruits inutiles, optimisant ainsi les performances et réduisant la latence avant l'envoi vers le module d'intelligence artificielle.
- **API Gemini** : analyse les commandes vocales transcrites à l'aide de techniques de traitement du langage naturel (NLP) et génère des réponses intelligentes adaptées au contexte.
- **Nœud de synthèse vocale** : convertit la réponse textuelle produite en signal audio via un moteur de synthèse vocale.
- **Haut-parleur USB** : restitue la réponse vocale générée, permettant une interaction naturelle avec l'utilisateur.

4.3 Intégration du microphone Spektra

Le microphone **Spektra** a été choisi pour sa simplicité d'intégration et sa compatibilité avec le Raspberry Pi 5 via une connexion USB directe. Cette solution permet d'éviter l'utilisation de cartes son supplémentaires, réduisant ainsi la complexité matérielle du système.

La configuration a été réalisée à l'aide des commandes Linux permettant de vérifier la détection du périphérique, de tester la capture audio et de valider la restitution sonore.

4.4 Développement du nœud de perception ROS2

Le nœud de perception constitue le point d'entrée logiciel du système vocal. Développé en Python, il assure la capture continue du signal audio et sa publication sur le topic

/audio/input.

Ce module intègre un mécanisme de VAD, permettant d’optimiser le traitement en filtrant les segments pertinents du flux audio. L’utilisation de ROS2 garantit une communication efficace et temps réel entre les différents modules.

4.5 Intégration de l’API Gemini

L’API **Gemini** a été intégrée comme moteur principal de traitement du langage naturel. Son choix repose sur sa faible latence et sa capacité à fournir des réponses contextuelles adaptées en temps réel.

Cependant, ce choix découle également d’une analyse comparative avec plusieurs approches alternatives.

Les architectures classiques basées sur une chaîne séquentielle *ASR (Automatic Speech Recognition) → NLU (Natural Language Understanding) → TTS (Text-To-Speech)* ont été envisagées. Bien qu’efficaces, ces solutions introduisent une latence cumulative importante, chaque étape nécessitant un traitement indépendant, ce qui réduit la fluidité de l’interaction en temps réel.

Des solutions basées sur des modèles d’intelligence artificielle locaux ont aussi été étudiées. Toutefois, elles exigent des ressources matérielles conséquentes (GPU, grande capacité mémoire) et présentent des performances limitées en compréhension contextuelle, notamment sur des plateformes embarquées.

À l’inverse, l’API Gemini Live propose une approche unifiée end-to-end, intégrant directement la détection vocale, la transcription, le traitement du langage naturel et la génération de réponse dans un pipeline unique.

Cette approche permet de réduire significativement la latence, d’améliorer la cohérence des réponses et de simplifier l’architecture globale du système.

Ainsi, le choix de l’API Gemini s’impose comme une solution optimale pour garantir une interaction vocale fluide, rapide et intelligente dans le robot AXEL.

Le pipeline de traitement se déroule selon les étapes suivantes :

1. Capture de la commande vocale.
2. Transcription du signal audio en texte.
3. Envoi de la requête à l’API Gemini .
4. Génération d’une réponse intelligente.
5. Publication de la réponse sur le topic /ia/decision.
6. Synthèse et restitution vocale.

Cette architecture permet une interaction naturelle et dynamique entre l’utilisateur et le robot.

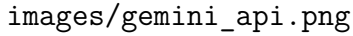


FIGURE 4.3 – Architecture de communication avec l’API Gemini Live [24]

4.5.1 Logique de décision et interprétation des commandes

Après le traitement du langage naturel par l’API Gemini, la réponse textuelle est analysée afin d’extraire une représentation intermédiaire appelée *intent*. Cette intention correspond à l’action que l’utilisateur souhaite que le robot exécute.

Le système repose sur un mécanisme de mapping entre les intentions détectées et les actions robotiques associées. Chaque intention est liée à un type d’action (`action_type`) et à une fonction spécifique (`function_name`) exécutée via le système ROS2.

TABLE 4.3 – Correspondance entre commandes vocales et actions

Commande vocale	Action exécutée
“ouvre la bouche”	Ouverture mâchoire
“ferme la bouche”	Fermeture mâchoire
“regarde à gauche”	Yeux à gauche
“regarde à droite”	Yeux à droite
“fais une expression joyeuse”	Expression faciale
Question	Réponse vocale

```
if intent == "ouvrir_bouche":
    publier("/axel/intent_action", IntentAction(
        action_type="jaw", function_name="ouvrir_bouche"))
elif intent == "fermer_bouche":
    publier("/axel/intent_action", IntentAction(
        action_type="jaw", function_name="fermer_bouche"))
elif intent == "regarder_gauche":
    publier("/axel/intent_action", IntentAction(
        action_type="eyes", function_name="regarder_gauche"))
elif intent == "regarder_droite":
    publier("/axel/intent_action", IntentAction(
        action_type="eyes", function_name="regarder_droite"))
elif intent == "expression_joie":
```

```
publier("/axel/intent_action", IntentAction(
    action_type="expression", function_name="expression_joie"))
else:
    publier("/audio/output", "réponse vocale")
```

Par exemple, la commande vocale « ouvre la bouche » est analysée par le module NLP et convertie en intention `ouvrir_bouche`. Cette intention est ensuite traduite en une action robotique via la publication d’un message ROS2 sur le topic `/axel/intent_action`, déclenchant l’ouverture de la mâchoire.

Cette approche permet de dissocier clairement la couche d’intelligence (NLP) de la couche de contrôle, garantissant ainsi une architecture modulaire, extensible et indépendante des modèles utilisés.

4.5.2 Interaction contextuelle basée sur l’identification de l’utilisateur

Afin d’améliorer la qualité de l’interaction homme-robot, le système AXEL intègre un mécanisme de personnalisation basé sur la reconnaissance faciale.

Le module de vision (*face_recognition*) permet d’identifier l’utilisateur en amont de l’interaction vocale. Lorsqu’un visage est reconnu, le nom de la personne est publié sur le topic `/axel/person_identity` et transmis au nœud `axel_live_node`.

Ce flux de données suit le pipeline suivant :

Caméra → Reconnaissance faciale → Identité → ROS2 → Intégration dans le prompt NLP

Le nœud `axel_live_node` intègre dynamiquement cette information dans le *prompt* envoyé à l’API Gemini, permettant ainsi d’adapter la réponse en fonction du contexte utilisateur.

Deux scénarios principaux sont distingués :

- **Utilisateur reconnu** : Lorsque l’identité est détectée, le système génère une réponse personnalisée, par exemple : “*Salam Ahmed*”. Cette approche renforce le caractère naturel et humain de l’interaction.
- **Utilisateur inconnu** : Si aucune identité n’est détectée dans le délai imparti, le système adopte une réponse générique, par exemple : “*Salam*”, garantissant ainsi la continuité de l’interaction.

Ce mécanisme repose sur une synchronisation entre les modules de vision et de traitement du langage naturel, permettant à AXEL de passer d’un système réactif à un système contextuel et intelligent.

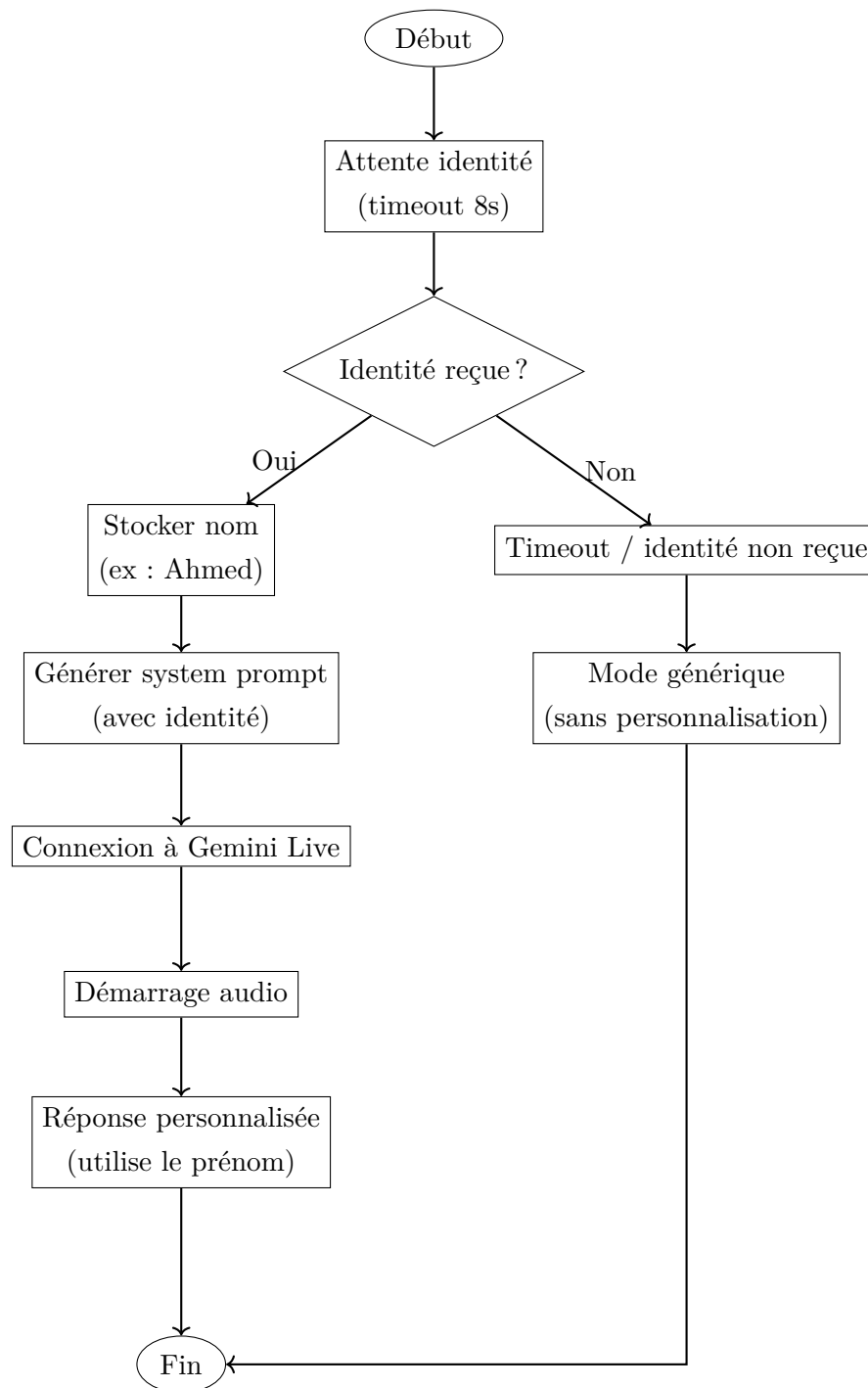


FIGURE 4.4 – Diagramme d’activité du processus d’identification et d’interaction intelligente du robot AXEL

Cette approche illustre la capacité du système à fusionner plusieurs sources d’information (vision et audio) afin d’améliorer la pertinence des réponses et l’expérience utilisateur globale.

4.5.3 Profils d’outils

Afin d’optimiser les performances du système et de contrôler la latence, trois profils d’outils ont été définis dans le fichier `config/tools.py`. Chaque profil correspond à un compromis entre richesse fonctionnelle et temps de réponse.

TABLE 4.4 – Profils d’outils et impact sur la latence

Profil	Nombre d’outils	Latence estimée	Usage
compact	5 outils	~ 1200 ms	Production — latence minimale
dynamic	~ 40 outils	~ 1800 ms	Développement — flexibilité accrue
full	~ 40 outils	~ 2000 ms	Debug — description complète des actions

Impact sur la latence : Chaque outil déclaré augmente le nombre de tokens envoyés au modèle, ce qui impacte directement le temps de traitement. En réduisant le nombre d’outils (profil compact), il est possible de diminuer la latence de 200 à 400 ms par interaction, améliorant ainsi la réactivité du système.

4.5.4 Gestion du multilinguisme

Le système AXEL supporte plusieurs langues afin de permettre une interaction naturelle avec l’utilisateur. Cette gestion multilingue repose sur un ensemble de règles définies dans le module de configuration.

Quatre langues principales sont prises en charge :

- **Darija tunisienne** : utilisée comme langue principale d’interaction. Une transcription latine standard est adoptée afin de représenter les sons arabes . Un vocabulaire cohérent est imposé pour garantir l’uniformité des réponses.
- **Arabe classique** : utilisé comme langue de substitution lorsque certains termes en darija ne sont pas disponibles ou ambigus. L’utilisation d’autres variantes dialectales est évitée afin de maintenir la cohérence linguistique.
- **Français** : utilisé avec une orthographe standard, incluant les accents, et sans mélange de registres (tutoiement/vouvoiement).
- **Anglais** : employé dans un style informel mais grammaticalement correct, avec l’utilisation naturelle de contractions.

Cette approche permet d’assurer une communication fluide, cohérente et adaptée au contexte linguistique de l’utilisateur.

4.5.5 Optimisation de la latence : Warm-up de session

Lors de l'établissement d'une nouvelle session avec la **Gemini Live API**, un phénomène de *cold start* est observé. Ce phénomène se traduit par une latence élevée lors des premiers échanges, pouvant atteindre environ 3000 ms, contre une latence moyenne de 1200 ms en régime normal.

Afin de réduire cet impact, une technique de *warm-up* a été mise en place. Elle consiste à envoyer une requête initiale minimale immédiatement après l'ouverture de la session, permettant ainsi d'initialiser le pipeline interne du modèle avant toute interaction utilisateur réelle.

Implémentation du warm-up

```
# Warm-up après connexion
async with client.aio.live.connect(...) as session:
    await session.send_client_content(
        turns=[Content(role='user', parts=[Part(text='.')])],
        turn_complete=True
    )
    await asyncio.sleep(1.5)

    while not self.speaker_queue.empty():
        self.speaker_queue.get_nowait()
```

FIGURE 4.5 – Implémentation du mécanisme de warm-up de session

L'effet du warm-up sur la latence a été évalué expérimentalement. Les résultats obtenus sont présentés dans le tableau suivant.

TABLE 4.5 – Impact du warm-up sur la latence

Tour	Sans warm-up	Avec warm-up
Tour 1	3079 ms	1420 ms
Tour 2	3019 ms	1380 ms
Tour 3	1840 ms	1200 ms
Tour 5+	1200 ms	1100 ms

Ces résultats montrent que le mécanisme de warm-up permet de réduire la latence du premier tour de **54%** (de 3079 ms à 1420 ms), améliorant ainsi la réactivité globale du système et l'expérience utilisateur.

Note technique : Le délai de `sleep` (1200 ms) a été déterminé expérimentalement à partir des mesures du Tour 2 (1380 ms) avec marge de sécurité, et peut nécessiter ajustement selon la charge réseau.

4.6 Intégration du haut-parleur USB

Le haut-parleur USB a été intégré afin d’assurer la restitution des réponses générées par le système. Sa connexion directe au Raspberry Pi simplifie l’architecture matérielle et garantit une sortie audio fiable et de bonne qualité.

4.7 Tests et validation

TABLE 4.6 – Évaluation des performances du module d’interaction vocale

Critère	Valeur mesurée	Condition de test	Observation
Détection du microphone	OK	Connexion USB directe (Raspberry Pi 5)	Détection immédiate sans configuration supplémentaire
Qualité du signal audio	Bonne	Environnement calme (distance ~ 0.5 m)	Signal stable avec faible bruit de fond
Taux de reconnaissance vocale	$> 90\%$	Commandes simples (darija/français)	Bonne précision, légère dégradation en environnement bruyant
Latence NLP (Gemini)	700 – 800 ms	Mode warm-up actif	Temps de réponse stable après initialisation
Latence initiale (cold start)	~ 3000 ms	Sans warm-up	Délai élevé lors de la première interaction
Latence optimisée (warm-up)	~ 1400 ms	Après warm-up	Réduction de plus de 50% de la latence initiale
Latence globale du pipeline	< 2 s	Pipeline complet (audio $\rightarrow$ réponse)	Compatible avec interaction temps réel

Les résultats expérimentaux obtenus confirment la capacité du système à assurer une interaction vocale fluide et réactive, avec des temps de réponse conformes aux exigences d’un système robotique temps réel.

Ces résultats mettent en évidence que l’architecture proposée permet d’établir un compromis efficace entre précision et temps de réponse, garantissant ainsi une interaction naturelle et performante en conditions réelles.

L’intégration du module VAD, associée au mécanisme de *warm-up* de session, a permis d’optimiser de manière significative la latence globale du pipeline, tout en maintenant un niveau élevé de précision dans la reconnaissance vocale.

Cette approche confirme la pertinence d’une architecture *end-to-end* pour les systèmes d’interaction vocale en robotique, en assurant une meilleure cohérence et une réduction des délais de traitement.

Cependant, les performances du système restent sensibles à certains facteurs externes, tels que le niveau de bruit ambiant, la distance entre l'utilisateur et le microphone, ainsi que la dépendance à la connectivité réseau pour l'accès à l'API Gemini. Ces éléments constituent des pistes d'amélioration pour les évolutions futures du système.

Conclusion

Ce chapitre a permis de concevoir et d'implémenter un système complet d'interaction vocale pour le robot **AXEL**, intégrant à la fois la capture audio, le traitement du langage naturel et la restitution vocale.

L'architecture proposée, basée sur la **Gemini Live API** et une intégration avec ROS2, a démontré sa capacité à assurer une interaction fluide, réactive et contextuelle, avec des performances compatibles avec les contraintes d'un système robotique temps réel.

Les optimisations apportées, notamment au niveau du VAD et du mécanisme de warm-up, ont permis de réduire significativement la latence et d'améliorer l'expérience utilisateur.

Cependant, la dépendance à la connectivité réseau constitue une limite importante, ouvrant la voie à des perspectives d'amélioration basées sur des solutions de traitement local embarqué.

Le chapitre suivant sera consacré à l'intégration des modules de vision par ordinateur et aux capacités de perception intelligente du robot.

Bibliographie

- [1] SoftBank Robotics, *NAO Robot*, <https://robotsguide.com/robots/nao>
- [2] SoftBank Robotics, *Pepper Robot*, <https://us.softbankrobotics.com/pepper>
- [3] Boston Dynamics, *Atlas Robot*, <https://www.bostondynamics.com/atlas>
- [4] Unitree Robotics, *Unitree G1 Humanoid Robot*, <https://www.unitree.com/g1/>
- [5] Kagermann, H., *Recommendations for Implementing the Strategic Initiative INDUSTRIE 4.0*, <https://www.acatech.de/publication/recommendations-for-implementing-the-strategic-initiative-industrie-4-0/>
- [6] R. Szeliski, *Computer Vision : Algorithms and Applications*, <https://szeliski.org/Book/>
- [7] Jurafsky, D. and Martin, J., *Speech and Language Processing*, <https://web.stanford.edu/~jurafsky/slp3/>
- [8] Agile Manifesto, <https://agilemanifesto.org/>
- [9] K. Schwaber and J. Sutherland, *The Scrum Guide*, <https://scrumguides.org/>
- [10] David J. Anderson, *Kanban Method*, <https://kanban.university/>
- [11] Corey Ladas, *Scrumban : Essays on Kanban Systems for Lean Software Development*, <https://leansoftwareengineering.com/ksse/scrumban/>
- [12] Microsoft, *Visual Studio Code*, <https://code.visualstudio.com/>
- [13] GitHub Inc., *GitHub*, <https://github.com/>
- [14] Open Robotics, *ROS2*, <https://docs.ros.org/>
- [15] Arduino, *Arduino IDE*, <https://www.arduino.cc/>
- [16] Canonical Ltd., *Ubuntu*, <https://ubuntu.com/>
- [17] Python Software Foundation, *Python*, <https://www.python.org/>
- [18] JGraph Ltd., *draw.io*, <https://www.drawio.com/>
- [19] Overleaf, *Overleaf*, <https://www.overleaf.com/>
- [20] Open Robotics, *RViz*, <https://docs.ros.org/en/humble/p/rviz2/>

- [21] Raspberry Pi Foundation, *Raspberry Pi 5*, <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [22] Espressif Systems, *ESP32*, <https://www.espressif.com/en/products/socs/esp32>
- [23] NXP Semiconductors, *PCA9685*, <https://www.nxp.com/products/PCA9685>
- [24] Google DeepMind, *Gemini API Documentation*, Disponible sur : <https://ai.google.dev>, consulté en 2026.
- [25] Tower Pro, *MG90 Micro Servo Motor*, <https://www.towerpro.com.tw/product/mg90s-3/>
- [26] Tower Pro, *MG90S Metal Gear Micro Servo*, <https://www.towerpro.com.tw/product/mg90s-3/>
- [27] Tower Pro, *MG996R High Torque Metal Gear Servo*, <https://www.towerpro.com.tw/product/mg996r/>
- [28] DSSERVO, *DS3218 Digital Servo 20kg*, <https://www.dsservo.com/en/products/DS3218.html>
- [29] DSSERVO, *DS3225 Digital Servo 25kg*, <https://www.dsservo.com/en/products/DS3225.html>
- [30] Spektra, *USB Microphone*, <https://www.amazon.com/s?k=spektra+usb+microphone>
- [31] *USB Speaker for Raspberry Pi*, <https://www.raspberrypi.com/products/usb-speaker/>
- [32] Bambu Lab, *Hyper PLA RFID Filament*, <https://store.bambulab.com/products/pla-hyper-series>
- [33] PickNik Robotics, *MoveIt Motion Planning Framework*, <https://moveit.ros.org/>
- [34] Bjarne Stroustrup, *The C++ Programming Language*, <https://www.stroustrup.com/>
- [35] InMoov Community, *InMoov Robot Head Design Discussion*, https://groups.google.com/g/inmoov/c/OTdm_bNyOCA, consulté le 22 avril 2026.
- [36] Meta, *React – A JavaScript library for building user interfaces*, <https://react.dev/>
- [37] Electron, *Electron Framework*, <https://www.electronjs.org/>
- [38] Three.js, *JavaScript 3D Library*, <https://threejs.org/>
- [39] Microsoft, *TypeScript Documentation*, <https://www.typescriptlang.org/>